

An Exploratory Usability Study of a Ballistic Calculations Visualizer

Evaluating Cognitive Load-driven UX Design Strategies

En explorativ användbarhetsstudie för visualisering av balistikberäkningar

En utvärdering av UX-designstrategier utifrån kognitiv belastning

---

Hugo Friberg <friberghugo@gmail.com>

Faculty of Health, Science and Technology

Master thesis in Computer Science

Second Cycle, 30 hp (ECTS)

Supervisor: Karl-Johan Grinnemo, <karl-johan.grinnemo@kau.se>

Examiner: Meiko Jensen <meiko.jensen@kau.se>

Karlstad, July 3rd, 2026

Abstract

User Experience plays a key role in the tools used to develop safety-critical systems. Poor usability increases the risk of user error and, in turn, the risk of significant consequences. In the domain of ballistic calculations, highly trained professionals use complex tools to develop products that meet strict standards. For such tools, effective interface design is necessary to minimize errors.

The transition from platform-dependent ballistic calculation modules towards a generalized ballistic calculation module has created the need for a standalone user interface. However, the lack of a dedicated visualization tool that fully meets the module's requirements has added complexity and reduced usability for professionals performing ballistic calculations.

In this thesis, three user interfaces based on different design approaches are developed and evaluated both quantitatively and qualitatively by representatives of the intended user group. To accomplish this, Cognitive Load Theory guides the development of the interfaces, and an exploratory usability study is conducted using the System Usability Scale and qualitative feedback.

The findings show that user interfaces that stress efficiency and flexibility yield higher perceived usability for the intended user group than those that prioritize learnability and guidance. At the same time, learnability-focused interfaces may still benefit novice users unfamiliar with the system. The findings, therefore, highlight the importance of balancing cognitive load, operational efficiency, user guidance, and flexibility when designing interfaces for complex, safety-critical systems.

Finally, the results suggest that there may not always be a single optimal interface for all users. Instead, interfaces that adapt to different experience levels or usage contexts may provide higher overall usability when both expert and novice users are part of the intended user group.

Keywords

UX, GUI, Usability, SUS, Human-Computer Interaction

Sammanfattning

Användarupplevelse spelar en avgörande roll i verktyg som används för att utveckla säkerhetskritiska system. Dålig användbarhet ökar risken för användarfel och därmed risken för allvarliga konsekvenser. Inom området ballistiska beräkningar använder högutbildade yrkespersoner komplexa verktyg för att utveckla produkter som uppfyller strikta krav. För sådana verktyg är en effektiv gränssnittsdesign nödvändig för att minimera fel.

Övergången från plattformsberoende ballistiska beräkningsmoduler till en generaliserad ballistisk beräkningsmodul har skapat behovet av ett fristående användargränssnitt. Avsaknaden av ett dedikerat visualiseringsverktyg som helt uppfyller modulens krav har dock ökat komplexiteten och minskat användbarheten för yrkespersoner som utför ballistiska beräkningar.

I detta examensarbete utvecklas och utvärderas tre användargränssnitt baserade på olika designmetoder, både kvantitativt och kvalitativt, av representanter från den avsedda användargruppen. För att uppnå detta används Cognitive Load Theory som vägledning i utvecklingen av gränssnitten och en utforskande användbarhetsstudie genomförs med hjälp av System Usability Scale och kvalitativ återkoppling.

Resultaten visar att användargränssnitt som betonar effektivitet och flexibilitet ger högre upplevd användbarhet för den avsedda användargruppen än de som prioriterar lärbarhet och vägledning. Samtidigt kan gränssnitt som fokuserar på lärbarhet fortfarande gynna nybörjare som är obekanta med systemet. Resultaten belyser därmed vikten av att balansera kognitiv belastning, operationell effektivitet, användarvägledning och flexibilitet vid utformning av gränssnitt för komplexa, säkerhetskritiska system.

Slutligen tyder resultaten på att det kanske inte alltid finns ett enda optimalt gränssnitt för alla användare. Istället kan gränssnitt som anpassar sig till olika erfarenhetsnivåer eller användningskontexter erbjuda högre övergripande användbarhet när både experter och nybörjare ingår i den avsedda användargruppen.

Nyckelord

Användarupplevelse, Grafiskt användargränssnitt, Användbarhet, SUS, Människa–datorinteraktion

Acknowledgements

I want to thank my supervisor, Karl-Johan Grinnemo, for supporting me throughout this thesis. Your mantra of always seeking opportunities for improvement substantially improved the quality of this thesis and the work underlying it.

I would also like to thank Reine Lundin, Tobias Lekare, and Isidor Karlsson at BAE Systems for their immense support and guidance.

Furthermore, I would like to thank Professor John Sören Petterson for guiding me on how to conduct user testing.

This thesis marks the end of the last five years of my university education. I would therefore like to thank all the friends and faculty at both Karlstad, and Ajou University that I've met along the way, and who have contributed to who I am now.

Acronyms

ArtSS Army Artillery School
CSS Cascading Style Sheets
DLL Dynamic Link Library
FMV Defence Material Administration
FNL Försvarets Normallufthav
FOF Field of Fire
GBM Generic Ballistic Module
GCS Global Coordinate System
GDPR General Data Protection Regulation
GUI Graphical User Interface
ICAO International Civil Aviation Organization
ISA International Standard Atmosphere
MDMA Michigan Department of Military Affairs
MDP Meterological Datum Plane
METCM Computer Meteorological Message
MOC Meta Object Compiler
MOS Meta Object System
MVC Model-View-Controller
NABK NATO Armaments Ballistic Kernel
NASA-TLX NASA-Task Load Index
NATO North Atlantic Treaty Organization
NFA No Fire Area
OPr Operational Processor
QML Qt Markup Language
S4 SG2 Shareable (Fire Control) Software Suite
SFL Safety Fire Limit
STANAG Standardization Agreement
SUS System Usability Scale
UEQ User Experience Questionnaire
UMUX-LITE Usability Metric for User Experience-Lite Version
UI User Interface
UX User Experience

Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Problem Description . . . 2
- 1.3 Thesis Objective . . . 2
- 1.4 Thesis Goals . . . 3
- 1.5 Ethics and Sustainability . . . 3
- 1.6 AI Disclosure . . . 4
- 1.7 Methodology . . . 4
- 1.8 Stakeholders . . . 4
- 1.9 Delimitations . . . 5
- 1.10 Outline . . . 5

2 Background and Related Work **6**
- 2.1 Ballistics and Fire Control Systems . . . 6
- 2.2 General Ballistic Module . . . 7
- 2.3 Cognitive Load and Usability in Complex Systems . . . 9
- 2.4 Usability Evaluation Methods . . . 12
- 2.5 Qt . . . 14
- 2.6 Related Work . . . 16

3 Design and Implementation **18**
- 3.1 Design Principles . . . 18
  - 3.1.1 Capabilities . . . 18
  - 3.1.2 Efficiency . . . 19
  - 3.1.3 Learnability . . . 19
  - 3.1.4 Satisfaction . . . 20
- 3.2 User Interface Design . . . 22
  - 3.2.1 Mandatory Inputs . . . 22
  - 3.2.2 Optional Inputs . . . 23
  - 3.2.3 Visualization . . . 25
- 3.3 System Architecture . . . 25
- 3.4 Implementation . . . 26
- 3.5 Comparison of Design Alternatives . . . 26
- 3.6 Design & Implementation Trade-offs . . . 28

4 Method and Result **29**

4.1 Method 29
4.1.1 SUS 29
4.1.2 Test Arrangement 30
4.1.3 Test Participants 33
4.1.4 Pilot Study 34
4.1.5 Test Execution 34
4.2 Results 35
4.3 Analysis 37

5 Conclusions and Future Work 42
5.1 Conclusion 42
5.2 Discussion 43
5.3 Future Work 44

Chapter 1

Introduction

When developing a tool for others, it is important to ensure users find it useful. User Experience (UX) testing is a common method for assessing how usable users perceive software [38]. This is important for any tool, but especially for safety-critical tools and pipelines.

One industry dealing with safety-critical software is the defense industry. Developing weapon systems is a meticulous and safety-critical ordeal where safety is paramount, as problems could mean the loss of life. BAE Systems develops artillery systems, and ensuring projectile accuracy is of utmost importance for safety.

The calculations required to maintain accuracy are complex, involve many parameters, and are cognitively demanding. The lack of dedicated tooling can introduce unnecessary cognitive load, increasing the risk of user error.

1.1 Background

The SG2 Shareable (Fire Control) Software Suite (S4) is a shared software library designed to implement standards for indirect fire[1] for North Atlantic Treaty Organization (NATO) members [2]. The S4 suite includes a module for calculating ballistic data, called the NATO Armaments Ballistic Kernel (NABK), as well as other modules that handle external factors, such as meteorological data and terrain clearance [10].

BAE utilizes a Generic Ballistic Module (GBM) that integrates and adapts the S4 suite for use with Swedish weapon systems, munitions, and safety procedures concerning fire limits[2].

When developing new equipment and munitions, ballistic characteristics can change [6]. To maintain accuracy, test firings are conducted to determine how the new equipment or munition performs and how environmental factors, such as wind,

[1] Projectiles fired in an arc to hit targets beyond line of sight.
[2] Geographical restrictions on where projectiles may be fired.

humidity, and temperature, affect the trajectory. An initial calculation is performed, followed by corrections based on test-firing results.

The GBM aims to generalize ballistic calculations for multiple weapon systems. Previously, simulators for individual weapons could be used as graphical interfaces. No such software exists for the GBM. And since the GBM aims to generalize calculations, none of the existing weapon-specific software is compatible.

1.2 Problem Description

Currently, there is no dedicated software for GBM prediction. This has led personnel to use the Archer artillery Graphical User Interface (GUI) to access the GBM, which, as a desktop application, is described as having a poor UI.

The GBM is a safety-critical piece of software, for which bugs and errors could have devastating consequences. Incorrect applications of safety limitations and ballistic parameters due to user error could result in the destruction of ecosystems, infrastructure, or, in worst case, loss of life.

To address this problem, this thesis aims to develop a C++ application using the Qt framework. This application will help personnel plan, conduct, and analyze test firings more effectively. Additionally, it will be used to showcase and develop the GBM's capabilities.

1.3 Thesis Objective

The purpose of this thesis is to investigate three GUI design philosophies and evaluate how well they support personnel in their work duties, affect their efficiency, and perceived usability.

Specifically, this thesis aims to investigate how three different GUI design philosophies, efficiency-driven, learnability-driven, and satisfaction-driven design, affect the perceived usability in an internal ballistics visualization application. This will be investigated through the following research questions:

- **RQ1:** How do efficiency-, learnability-, and satisfaction-driven GUI designs differ in perceived usability for the intended user group?
- **RQ2:** Which design philosophy is most suitable for the intended user group?

These design philosophies are based on established ISO usability characteristics. The reason for using three philosophies, and these specific philosophies, is due to time constraints and cognitive load, the variable we have chosen to evaluate the designs on.

1.4 Thesis Goals

This thesis aims to develop and evaluate three distinct design philosophies. Based on the results, an application will be created to enhance the user experience for company personnel. To accomplish this, a test group consisting of BAE employees and representatives from several government agencies will evaluate each design. Using a questionnaire, we will assess and compare the designs, identifying design features that both positively and negatively impact user experience.

The conclusions drawn from this thesis will help drive the development of the GBM visualizer, helping ensure the visualizer maintains a high level of usability throughout its development. We also hope to provide insights on design features that either increase or decrease perceived usability for complex systems.

1.5 Ethics and Sustainability

In 1998, the Michigan Department of Military Affairs (MDMA) released a report on a study investigating the environmental impact of an active artillery range from 1992 to 1995 [9]. The study aimed to analyze soil and groundwater samples from the Range 40 artillery and air-to-ground complex, the largest artillery firing range operated by the United States National Guard. After analyzing tens of thousands of soil and groundwater samples, the MDMA concluded that although metals and other residues from military activity were consistently identified above background levels, these levels remained below current regulatory limits. However, since the study was conducted over 30 years ago and under different regulatory jurisdictions, it is difficult to determine what conclusions Swedish regulators would reach today.

By providing users with an improved user experience, we can reduce user error. This, in turn, could lead to fewer test firings and, subsequently, reduced environmental impact.

As a part of this thesis, testers were recruited to evaluate the designs. When recruiting testers, there are several ethical considerations to take into account. Perhaps the most important ethical considerations are those of testers' privacy and their informed consent. Tester privacy focuses on ensuring confidentiality and minimizing the risk of testers being identified. This has a direct impact on testing: if a tester's identity can be linked to their answers, the tester might adjust their responses for fear of retaliation. This is especially pertinent for this study as the testing group is rather small, increasing the risk of tester identification. There is also a risk of workplace pressure, since testers' employers are stakeholders in the project.

There is also a legal aspect to consider. The EU General Data Protection Regulation (GDPR) imposes strong protections for how personal data must be handled [33]. In this thesis, the only personal data collected is an optional email address for those who would like to be notified of the thesis publication.

While informed consent from testers serves an important legal purpose, its primary purpose is to ensure testers are respected [7]. In recruitment, testers were informed that participation was voluntary, the goal of the study, and an estimate of the time each tester would need to dedicate to testing. In subsequent communication, special emphasis was placed on informing testers that there was no *right answer* and who to contact if they encountered problems.

1.6 AI Disclosure

The AI tool *Grammarly* was used while writing this thesis to improve the text's clarity.

1.7 Methodology

The application will initially be implemented using three distinct UX design philosophies:

- Efficiency-Driven (maximize information density and thus productivity)
- Learnability-Driven (minimize confusion and learning time)
- Satisfaction-Driven (maximize user satisfaction)

These design philosophies are derived from the software quality characteristics of Operability, Learnability, and User Interface Aesthetics [37].

BAE employees and government agency representatives will be invited to use and evaluate the three designs using a System Usability Scale (SUS)-based questionnaire. The SUS will be supplemented with two additional questions: an inter-design ranking and an optional free-text response. Although each question in the SUS can be analyzed individually, researchers recommend relying primarily on the final SUS score.

1.8 Stakeholders

This thesis project was conducted in collaboration with BAE Systems Bofors AB. The company develops state-of-the-art artillery systems, including the Archer Mobile Howitzer, the self-propelled anti-aircraft gun Lvkv 9040, and the L/70 naval artillery gun [3].

Test-firing new equipment is both expensive and harmful to the environment. A more effective tool could help reduce both BAE's costs and its environmental impact.

Several government agencies have expressed interest in this project. Some of these agencies include the Army Artillery School (ArtSS) and the Defence Material Administration (FMV), which is considering using the application.

1.9 Delimitations

The project has three main delimitations:

- Number of design philosophies
- Testing group
- Evaluation metrics

Limiting the design philosophies to three is primarily to remain within the thesis timeframe. ISO 25010, from which the chosen principles are derived, contains many more quality characteristics, and future work is encouraged to explore these further. Using these specific ISO quality characteristics enables mapping to cognitive load theory, which will be discussed in Chapter Two.

The testing group is limited to BAE personnel, representatives from ArtSS, and several government agencies. The GBM and S4 software applications, with which the application will interface are confidential. Using already-screened personnel eliminates the need for background checks for testers and restricts testing to the intended user group.

The SUS is a highly reliable metric for evaluating perceived usability. Including an inter-design ranking and optional free-text responses also eliminates the need to interview personnel after testing. Related work and conversations with experts have shown that the SUS is an appropriate tool with high reliability. SUS alone is sufficienct for determining the usability of each design since, according to experts, evaluating specific design features is best done by analyzing qualitative data. This is further explained in Chapter Four.

1.10 Outline

In chapter two, we will provide an introduction to key topics, including ballistics, methodology, and the tools used to implement our design philosophies. This will be followed by a section on the related works.

Chapter three will discuss the design and implementation of our three designs. We will begin by explaining how our methodology influenced design features. Continuing, we will discuss the GUI design, followed by the backend design, its implementation, and the differences between the two designs.

In chapter four, the evaluation of the designs will be discussed. A detailed description of the testing methodology and the results will be provided.

We will conclude with chapter five. This chapter will analyze the results presented in chapter four. It will discuss the lessons and insights revealed during the project and suggest future work.

Chapter 2

Background and Related Work

While this thesis studies UX design, the interdisciplinary nature of the application being developed necessitates an understanding of both ballistics and cognitive load theory. This chapter will begin by giving an introduction to the field of ballistics, followed by a short introduction to the GBM and its role in making ballistic calculations. Following this, cognitive load theory and the metric System Usability Score will be discussed. An introduction to the tool used to build the three applications will then be provided, followed by a discussion of notable related works.

2.1 Ballistics and Fire Control Systems

Ballistics is the study of what occurs to a projectile from the moment it is fired until it hits its target [16]. The subject is generally divided into two categories: *internal* and *external* ballistics.

Internal ballistics refers to the period from when the projectile is fired until it leaves the barrel. A mass, commonly referred to as a projectile, is loaded together with a propellant such as gunpowder. When the propellant is ignited, it deflagrates, expanding into a high-pressure gas that accelerates the projectile through the barrel. Some barrels are rifled, meaning the inside of the barrel has spiraling grooves. These grooves make the projectile spin, increasing its stability.

External ballistics studies how external forces, such as gravity, wind, and drag affect a projectile after it leaves the barrel. One of the most significant forces acting on a projectile is gravity. To compensate for gravity, the barrel can be raised so that the projectile will reach a certain altitude at a given distance.

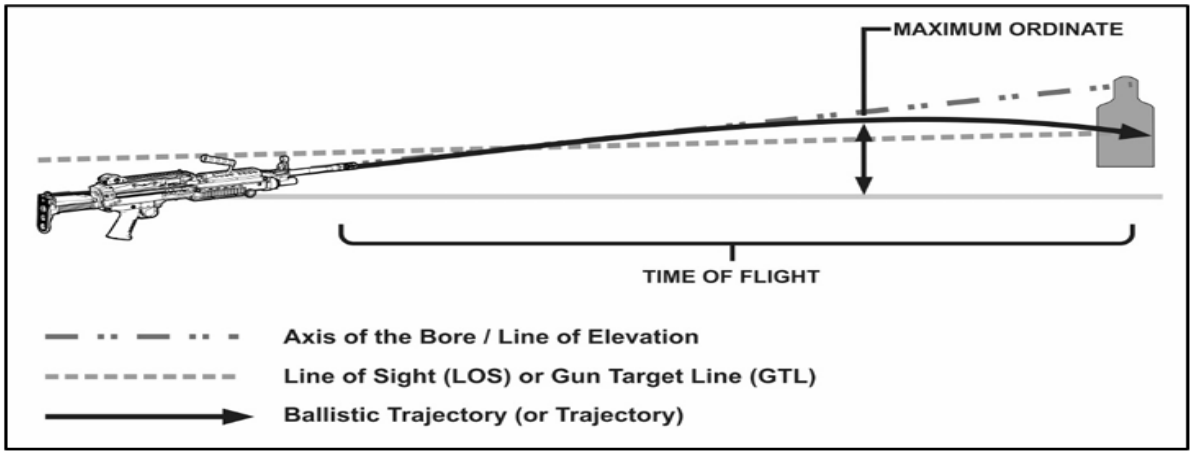


Figure 2.1.1: External ballistics. Source: US Army

When accounting for additional interdependent variables such as weather, fire limits, and terrain, ballistics calculations are inherently mentally demanding. The lack of dedicated tooling to visualize and clarify all these parameters complicates the mental model a user must construct to perform such calculations. While this may be manageable for experienced users, it can create a steep learning curve for less experienced users.

A visualization application must therefore help the user construct a clear mental model of ballistic calculations. It must be complex enough to allow modification of all parameters, while minimizing unnecessary complexity to reduce cognitive load.

2.2 General Ballistic Module

The GBM provides a common interface to both the S4 suite and additional modules. These additional modules are:

- S4 - NATO ballistics calculator
- NIFAK - estimates optimal amount and type of ammunition for a target
- Excalibur - S4 extension for the M982 Excalibur round
- BBL - Bofors Ballistic Library

The GBM is designed to be integrated into other programs. A program integrating the GBM is referred to as an Operational Processor (OPr). In this case, an OPr is any type of software utilizing the generalized ballistic calculation capabilities of the GBM to solve actual problems. This includes the application developed in this project as well as the weapon systems using the GBM. While weapon-specific OPrs exist, there is currently no generalized OPr for visualizing GBM calculations.

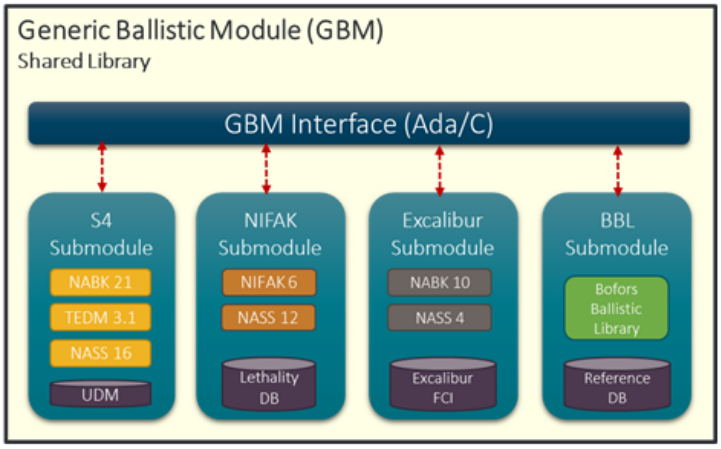


Figure 2.2.1: Overview of the GBM structure. Source: BAE Systems

Interaction with the GBM is done using a Dynamic Link Library (DLL). A dynamic link library (or shared library on Linux-based systems) is a collection of small programs and files that help other programs complete specific tasks [17]. These types of library files are linked to an executable either at compile time, in which case they are called *static libraries*, or at runtime, in which case they are called *dynamic libraries*. This difference is visualized in figure 2.2.2

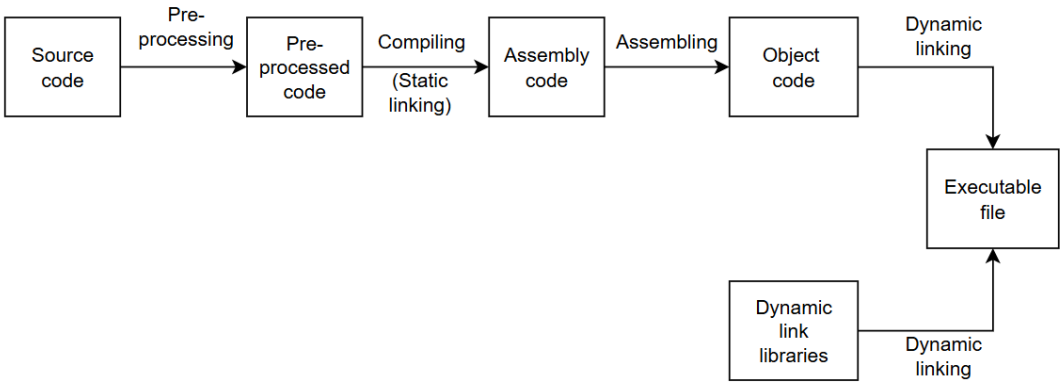


Figure 2.2.2: Static vs. Dynamic linking. Figure adapted from TechTarget

Since static libraries are linked directly into an executable, each executable needs its own copy of the library. Dynamic libraries on the other hand, are linked at runtime and can therefore be shared across multiple processes. This is advantageous from a development perspective, as less memory and fewer page switches are required when multiple processes use the library.

The GBM was not designed with GUI-based user interaction in mind but rather as a standalone backend module included in an OPr. This separation between the backend and frontend places significant responsibility on the OPr to provide a User Interface (UI) that presents ballistic data in a usable manner, thereby reducing the risk of user error.

2.3 Cognitive Load and Usability in Complex Systems

To evaluate and compare the perceived usability across multiple UX design philosophies, we need to introduce three different concepts: *Cognitive Load*, *Expert and Novice users*, and *Satisfaction*. In their thesis on design usability, N. V. Starbrink discusses the concepts of Cognitive Load [35]. Cognitive Load is ”the mental effort required to perform a task”. Cognitive load is usually divided into *intrinsic load*, the difficulty of solving the task itself, and *extraneous load*, which covers external distractions such as poorly designed instructional material, complex presentations, or a cluttered workspace.

Starbrink also discusses the concept of Expert and Novice users, highlighting differences in experience with a system. Expert users are described as individuals who frequently use a system and thus become highly familiar with it, possessing deep procedural knowledge and the ability to quickly recognize patterns. For these users, minimizing extraneous cognitive load is important to maintain efficiency. Contrast this with novice users, who, as discussed by Nielsen [25], have little (or no) experience with a system. Novice users rely heavily on explicit guidance, such as clear feedback and tutorials. While reducing extraneous load is generally beneficial, overly simplified designs may increase intrinsic cognitive load, making tasks harder to complete.

Nielsen also discusses the concept of satisfaction. They describe it as a combination between **Utility**: ”whether it [the system] provides the features you need”, and **Usability**: ”how easy & pleasant these features are to use”.

Cognitive load theory was originally developed by psychologist John Sweller [36]. Research on cognitive load theory by Sweller is extensive and spans a range of psychological phenomena, including the *Goal-Free Effect*, the *Isolated Elements Effect*, and the *Variability Effect*. For the purpose of this thesis, simply understanding the difference between intrinsic load and extraneous load is enough. Figure 2.3.1 depicts the three types of cognitive load developed by Sweller, of which only two will be utilized in this thesis.

The efficiency-driven design will aim to minimize intrinsic cognitive load by adopting a minimalist approach centered on core functionality, with minimal guidance. The efficiency-driven design should be ideal for the Expert User. The learnability-driven design will aim to minimize extraneous cognitive load. Introducing a user interface that guides the user step by step through the program, aiming to provide a deeper understanding of the components used in ballistic calculations.

Finally, the satisfaction-driven design will aim to provide the highest possible user satisfaction. For the purpose of this study, we define satisfaction as the inverse of total cognitive load. This is a simplifying assumption used to guide design features and does not represent a universally accepted definition.

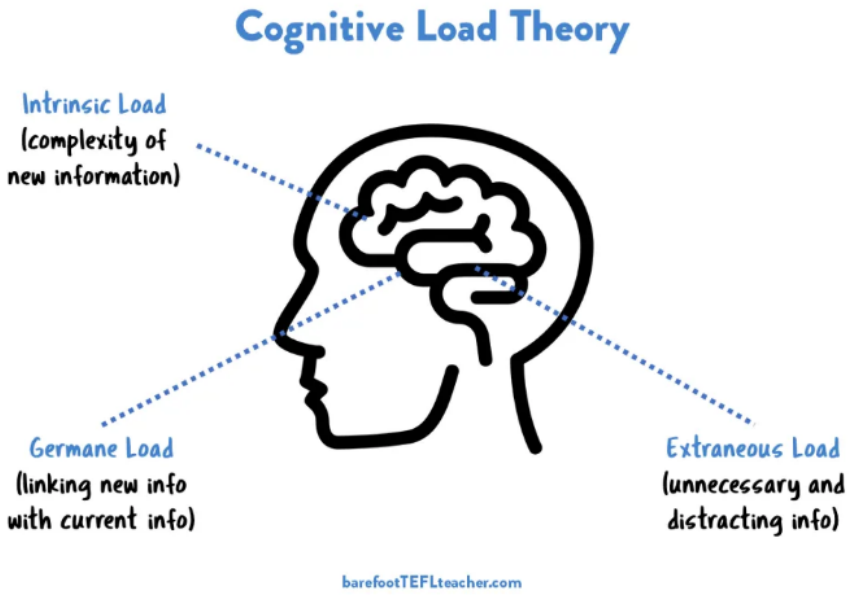


Figure 2.3.1: Depiction of Cognitive Load Theory. Source: barefootTEFLteacher

This means that the satisfaction-driven design will aim to minimize the overall cognitive load. While intrinsic and extraneous loads are not opposites, their relationship is reminiscent of an inverse relationship: reduced user guidance, and subsequently learnability, increases efficiency and reduces intrinsic load. Figure 2.3.2 shows the interaction between intrinsic and extraneous load. The blue line describes intrinsic load, the red line describes extraneous load, and the green line describes their sum.

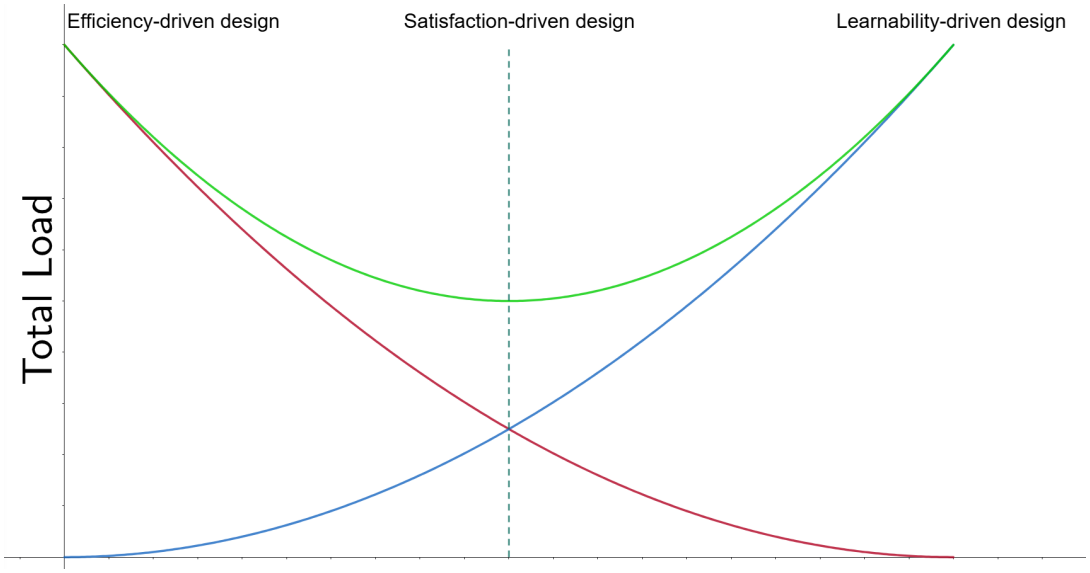


Figure 2.3.2: The interaction of intrinsic and extraneous load

Both the efficiency and learnability designs aim to minimize their respective types of cognitive load. This means that the optimal design likely lies somewhere between them. By attempting to make the satisfaction-driven design minimize the sum of intrinsic and extraneous load, we hope to create a much better picture of where the optimal design for the intended user group lies.

As we will discuss in the Related Works section, UX design features are highly context-dependent. This means that statements such as *"Design A yields higher a usability than design B*" are rarely universally applicable. However, analyszng existing UX principles can be a good first step.

One such set of principles are Nielsen's 10 usability heuristics. These ten principles are to be seen as "*broad rules of thumb*" rather than specific usability guidelines [23]. Nielsen describes the 10 principles as follows:

1. **Visibility of System Status**. The design should always keep users informed about what is going on, through appropriate feedback within a reasonable amount of time.
2. **Match Between the System and the Real World**. The design should speak the users' language. Use words, phrases, and concepts familiar to the user, rather than internal jargon. Follow real-world conventions, making information appear in a natural and logical order.
3. **User Control and Freedom**. Users often perform actions by mistake. They need a clearly marked "emergency exit" to leave the unwanted action without having to go through an extended process.
4. **Consistency and Standards**. Users should not have to wonder whether different words, situations, or actions mean the same thing. Follow platform and industry conventions.
5. **Error Prevention**. Good error messages are important, but the best designs carefully prevent problems from occurring in the first place. Either eliminate error-prone conditions, or check for them and present users with a confirmation option before they commit to the action.
6. **Recognition Rather than Recall**. Minimize the user's memory load by making elements, actions, and options visible. The user should not have to remember information from one part of the interface to another. Information required to use the design (e.g. field labels or menu items) should be visible or easily retrievable when needed.
7. **Flexibility and Efficiency of Use**. Shortcuts — hidden from novice users — may speed up the interaction for the expert user so that the design can cater to both inexperienced and experienced users. Allow users to tailor frequent actions.

8. **Aesthetic and Minimalist Design**. Interfaces should not contain information that is irrelevant or rarely needed. Every extra unit of information in an interface competes with the relevant units of information and diminishes their relative visibility.
9. **Help Users Recognize, Diagnose, and Recover from Errors**. Error messages should be expressed in plain language (no error codes), precisely indicate the problem, and constructively suggest a solution.
10. **Help and Documentation**. It's best if the system doesn't need any additional explanation. However, it may be necessary to provide documentation to help users understand how to complete their tasks.

In their article on applying Nielsens usaility heuristics in complex systems, Kaplan demonstrates how Nielsens principles almost always have to be adapted to the specific application and user group, or sometimes even discarded [14].

2.4 Usability Evaluation Methods

The System Usability Scale (SUS) is the most widely used tool for evaluating perceived usability in software [15]. Developed by John Brooke in 1996, the questionnaire consists of 10 statements [4]. Respondents rate their agreement with each statement on a scale from 1 (strongly disagree) to 5 (strongly agree). The SUS score is then calculated using the formula:

$$SUS = 2.5(20 + (Q_1 + Q_3 + Q_5 + Q_7 + Q_9) - (Q_2 + Q_4 + Q_6 + Q_8 + Q_{10}))$$

This calculation yields a score between zero and 100. Even-numbered questions are subtracted because they are negatively worded. With a sufficient number of respondents, SUS has demonstrated a reliability of 91%. The ten statements are shown in Table 2.4.1.

One weakness of SUS is that it cannot be effectively normalized. Research has shown that test scores can vary significantly across product types and interfaces. This can be seen in Table 2.4.2 when comparing products of varying complexity.

Another popular method of evaluating UX is the NASA-Task Load Index (NASA-TLX). It was developed in the 1980s by Sandra Hart at NASA and aims to measure the subjective workload of a given task [22]. Compared to SUS that measures perceived usability, the NASA-TLX measures perceived workload [8]. SUS also soley measures cognitive strain while the NASA-TLX accounts for physiological strain.

Statement #	Statement
1	I think that I would like to use this system frequently.
2	I found the system unnecessarily complex.
3	I thought the system was easy to use.
4	I think that I would need the support of a technical person to be able to use this system.
5	I found the various functions in this system were well integrated.
6	I thought there was too much inconsistency in this system.
7	I would imagine that most people would learn to use this system very quickly.
8	I found the system very awkward to use.
9	I felt very confident using the system.
10	I needed to learn a lot of things before I could get going with this system.

Table 2.4.1: The SUS question statements

Application	SUS Score
Excel	56.5
Word	76.2
Amazon	81.8
Google Search	92.7

Table 2.4.2: SUS scores for common applications [15]

The NASA-TLX is highly reliable and used in many safety-critical domains, such as Air traffic control, aviation, power plant management, and medicine. There are, however, some issues in using the NASA-TLX evaluation method in this thesis. Firstly, the NASA-TLX comprises six subscales that must be weighted manually. According to Hart, one of the main issues in using NASA-TLX is incorrectly weighing the subscales.

The second issue with using NASA-TLX in this thesis is that it is generalized across many tasks by including physiological strain, whereas SUS is specifically developed for evaluating software.

A common alternative for SUS is the User Experience Questionnaire (UEQ). Schrepp et al. compared the SUS with the shortened version of UEQ, the UEQ-S, and the Usability Metric for User Experience-Lite Version (UMUX-LITE) [18]. The researchers found

no significant difference between SUS and UMUX-LITE, concluding that they could be safely substituted.

When comparing SUS to the UEQ-S, the researchers identified differences in the results. Their conclusion for this was that the UEQ-S, unlike SUS, accounts for *hedonic* quality. Headonic quality can be described as encompassing more emotional metrics such as pleasure, joy, and novelty. Including headonic quality is undoubtedly leads to a better predictor of market adoption of a product, as it accounts for emotional responses in addition to the pragmatic-only evaluation provided by the SUS. However, because this project will develop an internal, problem-specific tool, headonic quality is less important. Using SUS over the UEQ-S brings no real benefit other than a more pragmatic-only perspective in evaluation.

The reason for choosing SUS over NASA-TLX and UEQ-S is due to the difficulty of manually weighing the NASA-TLX subscales, and that SUS provides a more pragmatic focus than that of UEQ-S.

2.5 Qt

Qt is a powerful C++ framework that enables straightforward cross-platform and GUI development [27]. It utilizes the Qt Markup Language (QML) language alongside its Meta Object Compiler (MOC) to create frontend components and connect them to backend C++ classes [31].

The framework enables connections between objects, called signals and slots [30]. When an object emits a signal, it triggers events in all objects connected through a slot. Qt also provides Unicode and networking support—features that even the most recent C++ standard lacks[1] [29], [5]. These capabilities make Qt a powerful tool for application development.

The MOC is part of the larger Meta Object System (MOS). This system includes various macros that inform the MOC about which files should be treated as Qt objects and specify their types.

Three common macros are Q_OBJECT, QML_ELEMENT, and Q_INVOKABLE. The Q_OBJECT macro instructs the MOC to enable Qt-specific functionality for a class, such as signals and slots. Adding QML_ELEMENT to a class or struct allows front-end QML code to reference it. Q_INVOKABLE can be used to mark a function within a class that already contains a Q_OBJECT macro, allowing that function to be called from front-end QML code.

[1]Most recent release as of writing is C++23

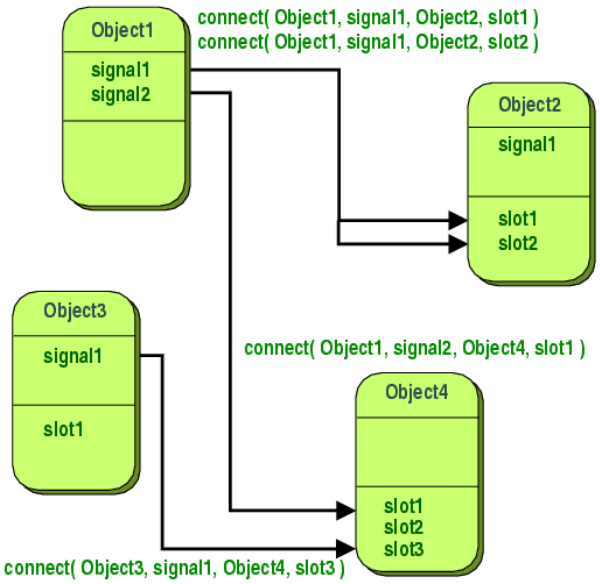


Figure 2.5.1: Signals and slots. Source: Qt documentation [30]

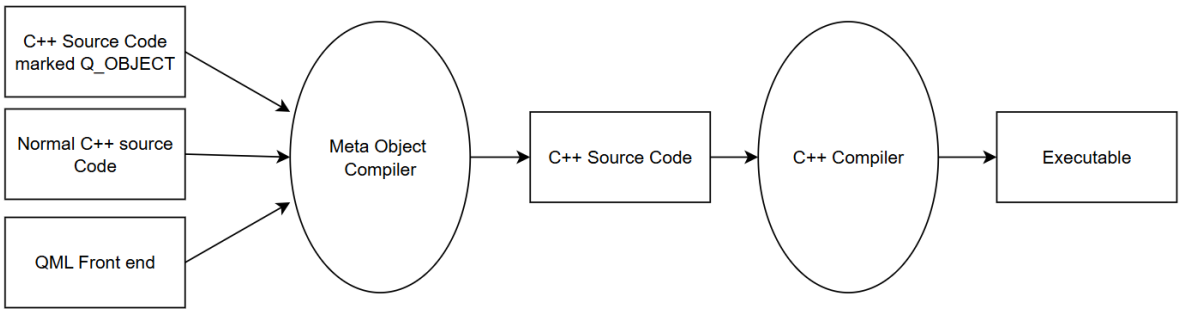


Figure 2.5.2: Qt build process. Figure self-made

QML is described as a ”multi-paradigm language that enables objects to be defined in terms of their attributes” [28]. It has a structure similar to the Cascading Style Sheets (CSS) language used in web development, while also supporting JavaScript for additional functionality. QML objects also support signals and slots (in QML called signals handlers), allowing QML objects to natively support communication to a C++ backend.

Other than Qt, the tool Dear ImGui was also considered. Dear ImGui is an open-source C++ tool for creating cross-platform desktop applications. The reason for choosing Qt over Dear ImGui lies in the level of complexity. While both tools offer largely the same functionality, Dear ImGui requires more aspects to be handled explicitly, whereas Qt generally allows explicit control while also providing default presets. Examples of where Dear ImGui requires explicit handling are error handling and selecting a platform-appropriate rendering pipeline.

Qt QML also offers a more readable way to build GUIs than Dear ImGui, which expresses everything in C++.

```
Rectangle {
    width: 100
    height: 100
    color: "red"
}

Button {
    id: fibonacciBtn

    function fibonacci(n) {
        var arr = [0, 1]
        for (var i = 2; i < n + 1; i++)
            arr.push(arr[i - 2] + arr[i - 1]);

        console.log(arr)
    }

    //signal handler for the 'Clicked' signal native to the Button object
    onClicked: {
        fibonacciBtn.fibonacci(10)
    }
}
```

Listing 2.1: Example of QML objects

2.6 Related Work

Researchers at the University of Botswana evaluated the usability of an E-government system in 2022 [32]. Using SUS and coding from interviews, they calculated a SUS score of 52.4, indicating poor usability. With this score and the interviews, the researchers successfully identified specific root causes and suggested improvements to the system. This research shows that combining SUS with interviews can yield conclusive, reliable results. While the E-government system is designed for a wide range of users, the target group in this thesis is smaller. As a result, more weight can be given to individual free-text responses, as each tester represents a larger portion of the intended users.

The study demonstrates how the practical application of SUS combined with qualitative feedback can provide valuable insights. However, the evaluated system was fully developed, limiting the usefulness of the study's conclusions due to software ossification. In contrast, this thesis applies a similar approach, combining SUS with qualitative feedback in an exploratory setting earlier in the development process. This means that the conclusions of this thesis could be used to steer development and directly improve usability, rather than simply evaluating the application's usability.

In their dissertation on developing empirical frameworks for UX evaluation, Mahlke discusses the state of non-empirical evaluation frameworks [19]. Mahlke notes that for

non-empirical evaluation frameworks, differing definitions of concepts and sometimes even contradictions are not uncommon. This highlights the complexity of UX as a concept, and challenges the notion that certain design features always have a consistent effect on UX metrics.

While noting the lack of empirical frameworks, Mahlke continues to say that using differing concept definitions and non-empirical evaluation frameworks does not mean that such methods lack merit. Mahlke instead points to several studies indicating that UX metrics are often highly context-dependent and that a context-aware, user-oriented approach is critical. Mahlke's conclusions support our choice to use nonstandard definitions for *efficiency*, *learnability*, and *satisfaction*. Conducting an exploratory study, followed by an iterative development after the thesis, will help provide (and maintain) relevant context and user preferences, following Mahlke's recommendations. A detailed description of exploratory studies is given in section 4.1.2.

In their paper on UX in complex systems, Savioja et. al. explore UX in the nuclear industry [26]. As part of a project to modernize the control rooms of nuclear power plants, the researchers explored the importance of UX in highly complex systems, operated by highly trained professionals. Of their many findings, two are important to this study. Firstly, the longer operators used a tool, the higher the score that tool received. This aligns with one of Mahlke's conclusions that usage frequency correlates with higher UX testing scores. This matters for interpreting the results of this study. As this thesis develops a completely new tool, SUS scores might be negatively affected by both the tool's actual usability and its novelty.

Secondly, researchers theorized that professionals tend to be more neutral in their scoring than the population at large. The argument for this is that professionals have both a more nuanced view and a deeper technical understanding than the average person. This could be a limitation for an exploratory study developing an internal tool, with a relatively few testers. In an exploratory study, it is important to maximize the information you obtain from each tester. And, as this is an internal, highly specialized tool, the risk of receiving overly neutral answers could make drawing conclusions difficult.

Chapter 3

Design and Implementation

We begin by describing the design rationale behind each interface, including how theoretical concepts have informed key design features. The chapter then outlines the overall system architecture and the technical implementation, focusing on how the designs were realized using the Qt framework. Finally, we compare the three implementations, highlighting their common components, key differences, and how the development evolved across iterations.

Details regarding the CMake build process and application entry point are not central to this chapter and are therefore provided in Appendix .1.

3.1 Design Principles

This section presents the design principles that guided the development of the UI. Design is based on cognitive load theory and explores different design principles across the three designs. First, the capabilities, i.e., what the user should be able to do, will be explained. This will be followed by a description of the design principles for each design.

3.1.1 Capabilities

Two types of calculations can be performed by the GBM. The first calculation type is called a **firing solution**. A firing solution takes as input the positions of the weapon and target, the specific munition type used, and optional data such as weather, terrain, and fire limits. The firing solution then returns the direction (azimuth) and barrel angle (elevation) needed to hit the specified target.

The second type of calculation is referred to as a **trajectory calculation**. Trajectory calculations take as input weapon position, azimuth, elevation, optional weather data, and a munition type. The calculation then returns the location where the munition will land. Trajectory calculations primarily exist to verify firing solutions.

The designs should be capable of performing both types of calculations and displaying the results. Results should include information such as total height and distance, azimuth, elevation, munition used, time of flight, and angle of fall at impact. The designs should also graphically display the trajectory path of calculations.

Furthermore, the designs should be capable of switching between displaying results in degrees and in MILs (miliradians), a unit often used in military contexts.

3.1.2 Efficiency

The efficiency-driven approach seeks to reduce intrinsic cognitive load through a minimalist interface that offers limited visual guidance. In this layout, the application presents all functionalities to users in a single view (page) as soon as it opens (see Figure 3.1.1).

Input fields are positioned at the top of the window, followed by a section that displays calculation results and warnings. Beneath this, a graph visualizes the calculated trajectory, showing both the total horizontal distance traveled and the maximum height reached. At the bottom, a control bar enables users to select the calculation type and result units.

The layout follows a top-to-bottom, left-to-right structure, mirroring the natural reading order. It starts with weapon and target locations, continues with the weapon hierarchy tree discussed in Section 3.2.1, and concludes with optional files. This sequence aligns with the logical flow of user interaction.

The efficiency-driven design offers minimal feedback on input validity, emphasizing speed and simplicity over user guidance. For instance, users cannot select Computer Meteorological Message (METCM) as a weather option unless a custom weather file is loaded. Other than the absence of a selectable action, there is no indication if a file has been parsed or loaded.

3.1.3 Learnability

The learnability-driven design aims to minimize extraneous cognitive load by providing increased visual guidance for the user. The application is organized into multiple views, each addressing a specific stage of the calculation process (see Figure 3.1.3).

To guide users through the calculation process, the interface is structured as a checklist highlighting the various elements that can be specified. Once a user enters valid input in a view, the associated checklist item is marked with a green checkmark, signifying completion of that step. Input fields clearly indicate valid values to help users understand acceptable inputs. For certain multiple-choice fields, images of the selected option are displayed to provide a more visual indication of the user's choice.

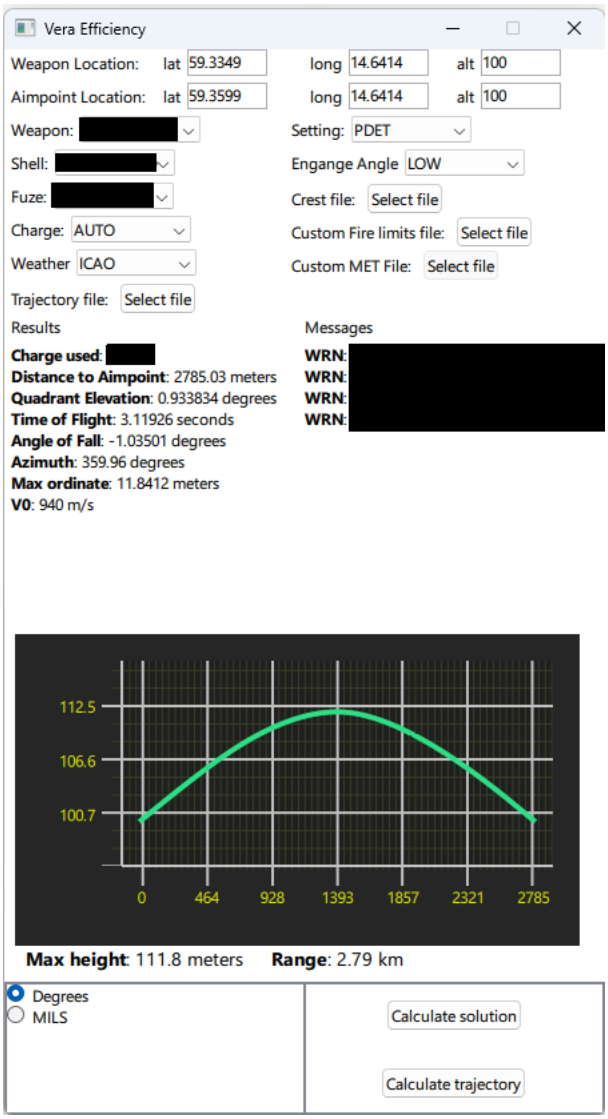


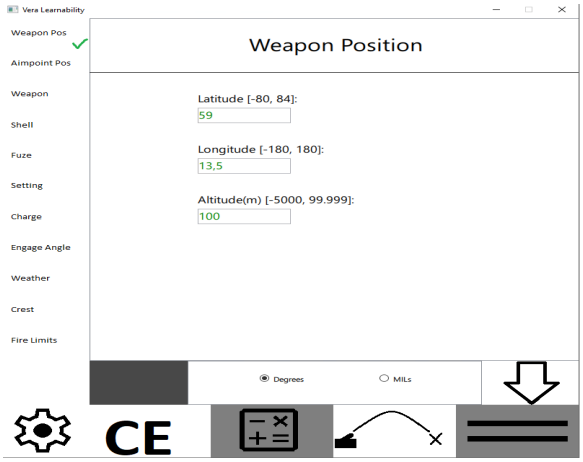
Figure 3.1.1: The efficiency-driven design

When a step is validated, visual cues such as arrows and checkmarks appear to indicate that the user can move to the next stage. Users can also navigate through the checklist using two arrow buttons.

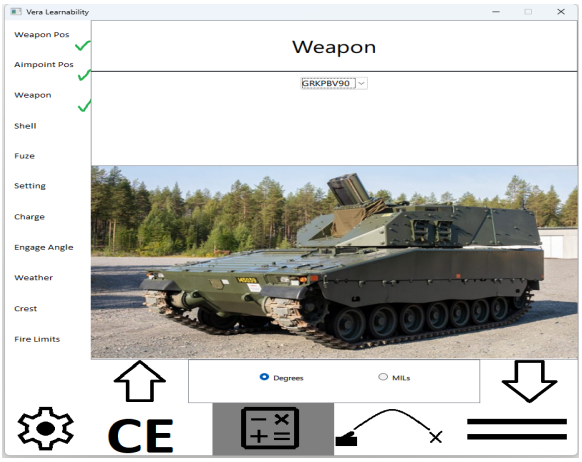
Some inputs are either standards or based on parsed text files. To further clarify what data is used for calculations, the application displays either the chosen standard or the parsed file in an easy-to-read format. When all required steps are complete, the user can press the equal sign button to be taken to the result view. This view is similar to what is shown in the efficiency-driven design.

3.1.4 Satisfaction

The satisfaction-driven design seeks to minimize both intrinsic and extraneous cognitive load. Inspiration for this approach was drawn from efficiency-driven and learnability-driven methods (see Figure 3.1.4).

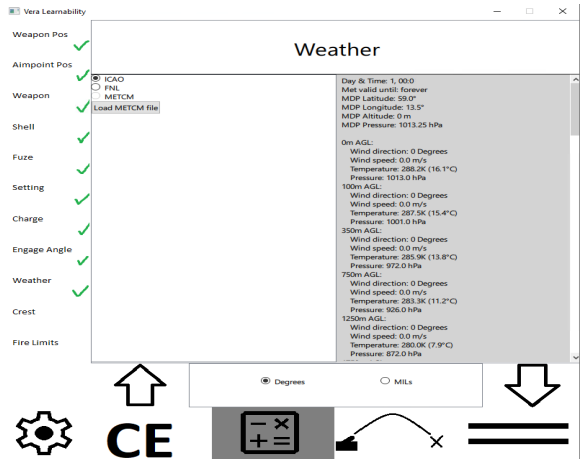


(a) Weapon position view

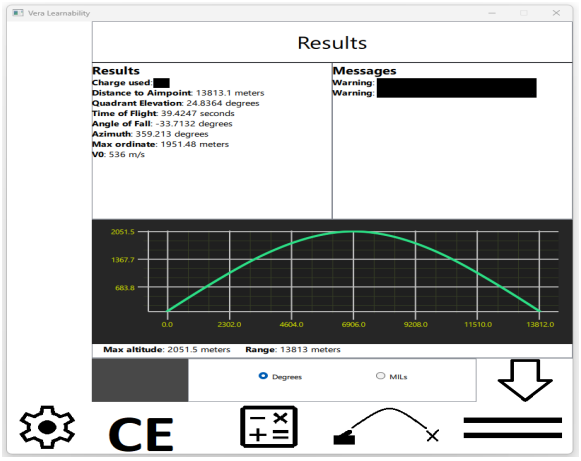


(b) Weapon selection view

Figure 3.1.2: Two input views in the learnability-driven design. Full sized figures are available in Appendix .9.



(a) Weather selection view



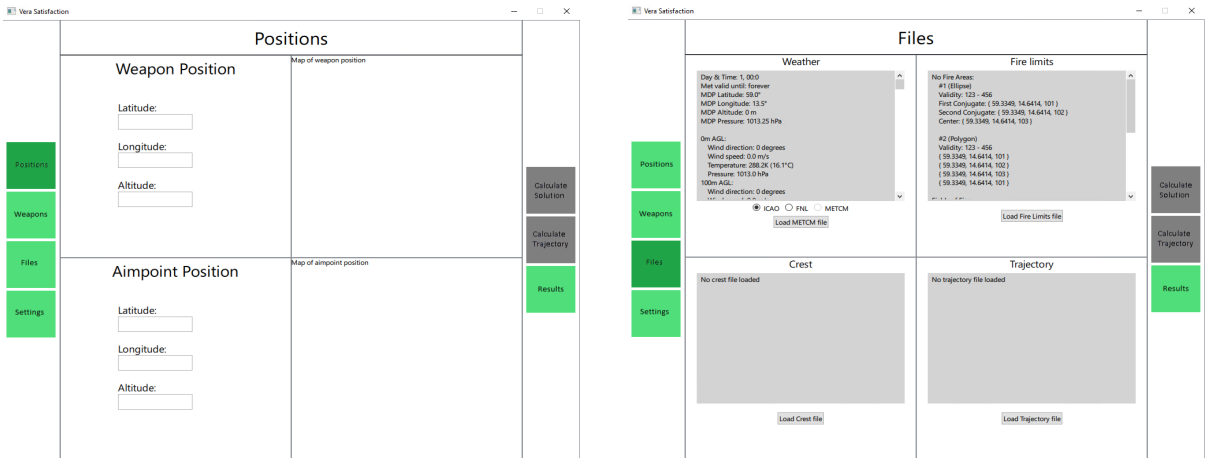
(b) Results view

Figure 3.1.3: Weather and Result views in the learnability-driven design.

This design builds on learnability-driven concepts and incorporates multiple views. Unlike the learnability-driven approach, however, the satisfaction-driven design organizes these views into broader categories rather than assigning one view per task. The categories used are position-based, weapon-based, and file-based inputs.

Additionally, views were separated based on whether they handled input or output data. Input view buttons were positioned on the left side of the window, while output views were placed on the right. This arrangement reflects the natural left-to-right reading flow.

Once all required inputs for a calculation are provided, the corresponding calculate button on the right transitions from grey to green. Pressing this button performs the selected calculation and displays the results in the results view, which is consistent with the views in the other designs.



(a) Weapon and target position view (b) Files view

Figure 3.1.4: Two input views in the satisfaction-driven design

Unlike the efficiency- and learnability-driven designs, the satisfaction-driven design allocates space for map views within both the Positions and Results views. Although this feature was planned, it was not implemented due to time constraints. The categorization in the satisfaction-driven design allowed for ample map space, whereas the efficiency- and learnability-driven designs intended to use a secondary window for maps.

The design principles discussed here serve as the foundation for the UI designs presented in the following sections.

3.2 User Interface Design

This section describes the user interface design, including how users enter input parameters and how results are presented.

3.2.1 Mandatory Inputs

The user interface provides input fields to specify the geographical coordinates of both the firing artillery and the target. These positions are specified using the Global Coordinate System (GCS) and are stored as a struct of three doubles: latitude, longitude, and altitude. All data and operations related to geopositioning are managed by the *LocationManager* class. An example of a GCS point is provided in the appendix.

After selecting the weapon and target locations, the user chooses the weapon type, projectile, and propellant. These options are organized hierarchically: some weapon systems allow different sets of projectiles, which in turn enable specific propellant options. On startup, available weapon systems and their subcomponents are loaded from the GBM and shown to the user. When a selection is made, the chosen component

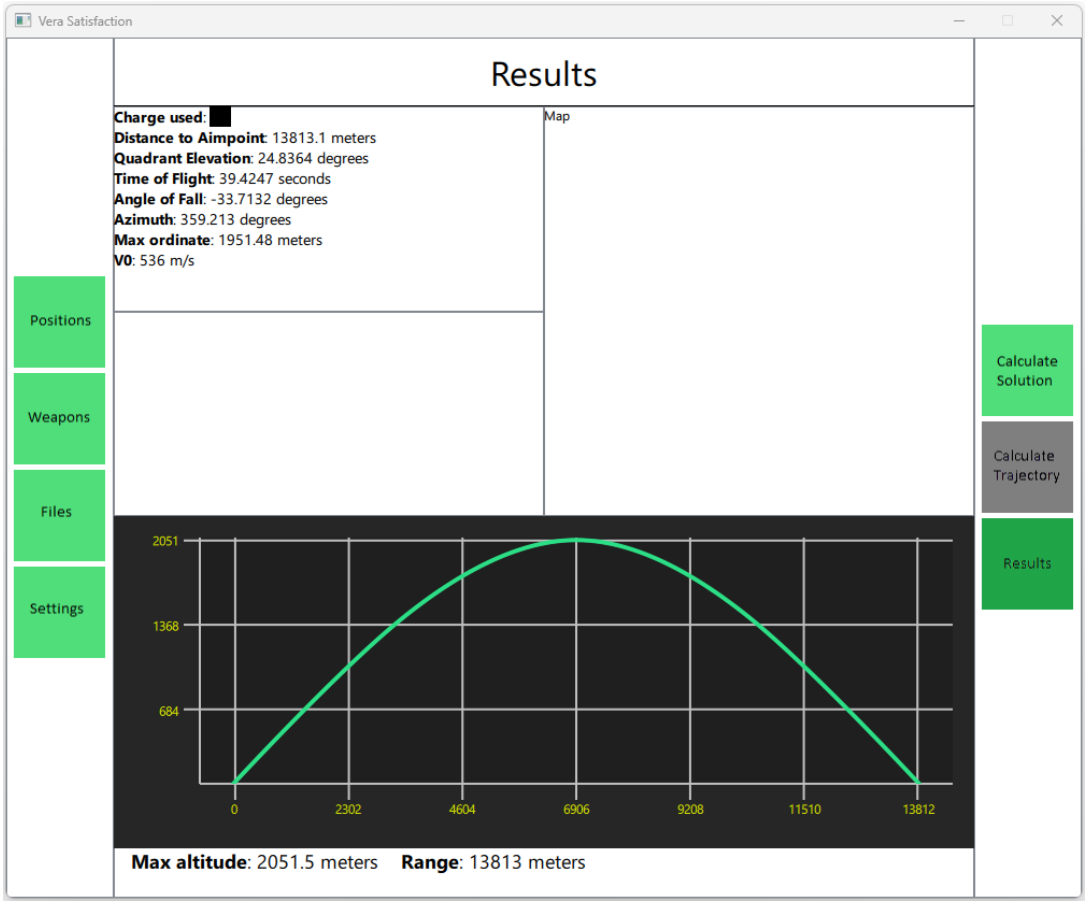


Figure 3.1.5: The satisfaction-driven design's result view

and its children are reloaded. Weapon-related data and operations are managed by the *WeaponManager* class. The user may also select the engagement angle: either **HIGH**, producing a high trajectory arch, or **LOW**, resulting in a flatter trajectory closer to the ground. The engagement angle is managed by the *SettingsManager* class.

3.2.2 Optional Inputs

The user interface allows users to provide optional inputs that can enhance the safety or accuracy of calculations, or enable additional functionality. There are four types of user-defined inputs, each provided as a text file and parsed by their respective classes: custom weather, fire limits, crests, and trajectory. These input types will be discussed in the following sections. Examples of these text files are shown in the appendix.

The International Civil Aviation Organization (ICAO) and Försvarets Normallufthav (FNL) standard atmospheres are, by definition, generalized. However, important parameters such as temperature, wind, and air pressure can vary drastically depending on location and altitude. Temporary weather stations, such as weather balloons and specialized aircraft, or permanent stations like those at airports, allow measurement of local wind, temperature, air pressure, and more to increase accuracy.

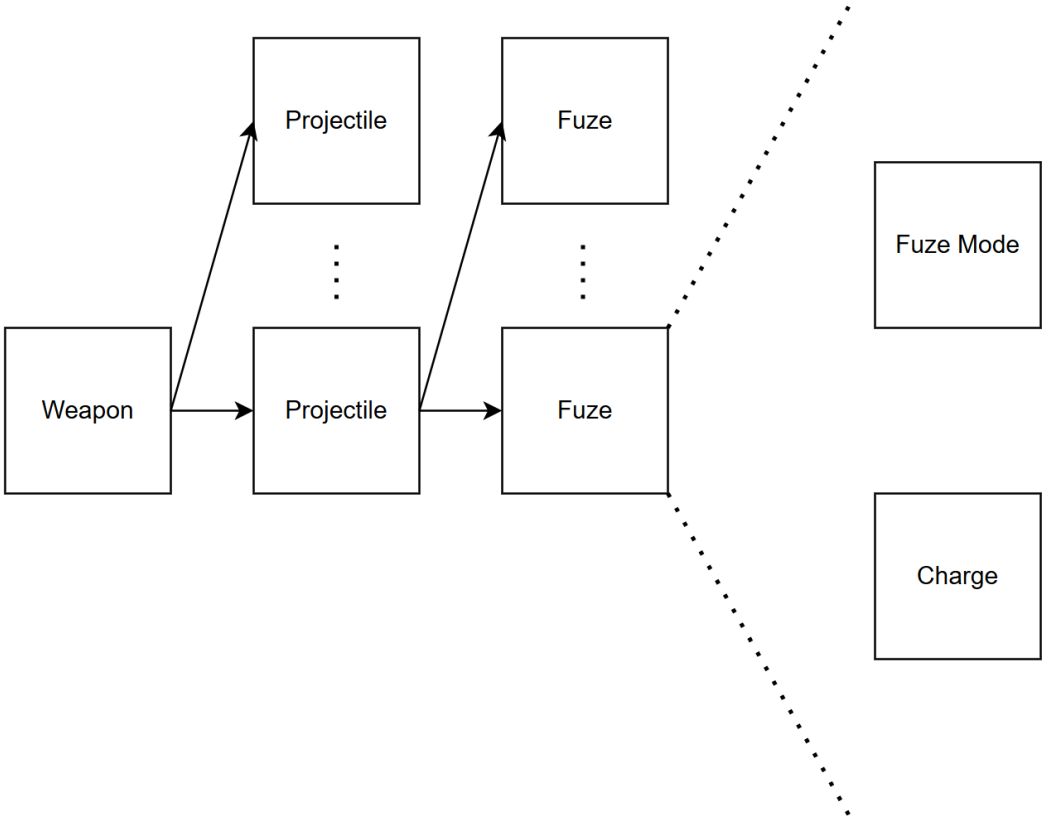


Figure 3.2.1: Weapon hierarchy tree. Figure self made

Fire limits serve a critical safety role. Whether in armed conflict or peacetime, preventing injury and destruction is paramount. In the application, fire limits are provided in three formats: *Safety Fire Limit (SFL)*s (sectors defined by an angle and distance relative to the weapon), *No Fire Area (NFA)s* (explicitly prohibited firing zones), and *Field of Fire (FOF)s* (explicitly permissible firing zones).

Crests serve a related but distinct purpose from fire limits. They account for terrain-based obstacles such as trees and mountains, playing an important safety role by preventing personnel from inadvertently aiming at nearby obstacles rather than distant targets. Additionally, crests serve operational, economic, and environmental functions. Failing to account for terrain could result in missing the intended target, wasting expensive rounds, and potentially damaging the local ecosystem at the impact site.

Trajectory presets have a less direct operational use. They calculate a trajectory path based on barrel elevation, direction, and projectile characteristics. These presets can be used to independently verify calculations and refine how different projectiles respond to changes in temperature, wind, and initial velocity, potentially providing significant economic benefits.

The formats of these text files differ. Custom weather files are comma- and line-separated, whereas fire limits and crests incorporate labels to clarify their scope.

Trajectory files use a *key=value* format rather than comma separation. This distinction exists because custom weather and trajectory files each contain a single bundle of information, while crests and fire limits contain multiple bundles. Providing different formats also gives testers several options for feedback.

3.2.3 Visualization

After entering all required inputs, the user can choose to either calculate a firing solution or run a trajectory calculation. For both options, GBM functions are called to load all user selections into the GBM. Following certain GBM function calls, a list is returned containing information, warning, and error messages. This list is checked after each function call for critical errors and is then tabulated for user display alongside the calculation results. Although the target location is required only for firing solution calculations, it is also loaded for trajectory calculations to avoid a known bug in which GBM emits a warning if no target location is provided. The user can also specify whether to display the results in degrees or mils.

The results from the GBM are presented in either two or three sections, depending on whether the user selected a firing solution or a trajectory calculation. Both options provide a list of error and warning messages generated by the GBM, which may include issues such as the projectile crossing a safety limit or the METCM being out of date. The second section presents the actual calculation results, including the maximum projectile height, impact location, initial velocity, and angle of fall at impact. If the user chooses to calculate a firing solution, an additional section displays a graph of the projectile trajectory, including maximum height and total distance.

3.3 System Architecture

Good software design is essential for large, complex systems [34]. This project adopts the Model-View-Controller (MVC) design pattern [21]. The View consists of QML files that interact with a controller C++ class instantiated as a QML_SINGLETON. The controller communicates with several C++ modules, each handling a specific task, such as parsing weather files or identifying safe firing zones. This modular approach not only allows independent development of each module but also facilitates the addition of new features in the future.

In addition to data from the GBM, the application relies on user-created text files, primarily for weather and safety limits. Standard weather formats such as the ICAO International Standard Atmosphere (ISA), and the Swedish armed forces' equivalent, FNL, are supported. Custom formats like METCM can provide location-specific weather data [13]. The GBM also enables users to define different types of safety areas, specifying where projectiles may or may not be fired. Allowing users to create their own presets adds further modularity and flexibility to the application.

3.4 Implementation

As mentioned in Chapter Two, QML is similar to CSS and JavaScript. The frontend, serving as the view component, manages user interaction, accepting input and displaying output. Communication between the backend and frontend occurs in two ways. To communicate from the frontend to the backend, Q_INVOKABLE functions are used. These are functions whose declarations are prefixed with the Q_INVOKABLE keyword, making them accessible to QML. Communication from the backend to the frontend is handled with Qt properties, which allow you to expose a variable to QML along with a signal that notifies QML when the variable is updated.

For example, consider a list of available projectiles. You can expose this list using a Qt property, and whenever you want the frontend to reflect changes in the list, you simply emit the associated signal.

```
1 Q_PROPERTY(QList ProjectileList MEMBER m_projectileList NOTIFY
     projectileListChange)
2 ...
3 m_projectileList.append(<new projectile>);
4 emit projectileListChange();
```

Listing 3.1: Example of how Qt properties are used

Frontend development primarily involved combining layouts, text fields, buttons, and other existing QML objects. Frequently used object combinations, such as a label with a text box, were encapsulated in custom objects to improve readability.

As mentioned in Section 3.2.3, a graph of the projectile trajectory is shown when the user calculates a firing solution. The Qt *GraphsView* object is used for this purpose. By sampling the starting, maximum, and final altitudes, as well as the total distance from the calculation, three 2D coordinates are generated. These coordinates are passed to the *GraphsView* object, which creates a cubic Bézier curve (a line with continuous first and second derivatives) and displays it on a graph for the user.

3.5 Comparison of Design Alternatives

In the efficiency-driven design, the entire frontend was developed within a single view (page). In contrast, the learnability- and satisfaction-driven designs used multiple views. This was achieved using the QML Loader object, which allows a source QML file to be loaded dynamically. An enumeration declared in the backend, exposed via the Q_ENUM and Q_PROPERTY keywords, enables the backend to trigger a view change by updating a variable and emitting a signal. For the learnability-driven design, this mechanism permits, depending on whether a firing solution or trajectory is being calculated, the complete disabling of views containing the aimpoint location input or the trajectory file.

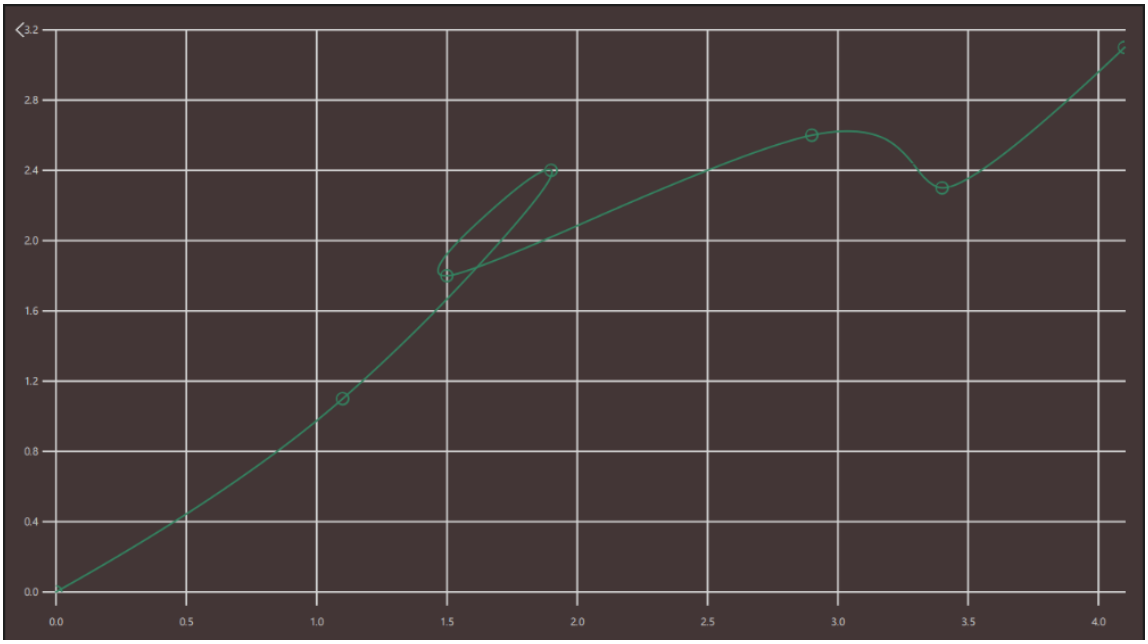


Figure 3.4.1: Example of a cubic Bézier curve. Source: QML documentation

The efficiency-driven design is a monoview application, whereas the learnability- and satisfaction-driven designs are multiview applications. This segmentation enabled the application to be divided into logical parts. In the learnability-driven design, the application was split into small, incremental steps. The satisfaction-driven design divided the application into distinct areas of responsibility: locations, weapon settings, files, settings, and results. The segmentation into multiple views aims to introduce an efficiency-learnability tradeoff. As the number of views increases, each view becomes easier to understand, but efficiency declines as the time spent switching between views increases.

Two further distinctions between the applications are input validation and how progression is indicated to the user. In the efficiency-driven design, minimal feedback is provided: if an error occurs, either nothing happens, or feedback is provided through GBM warnings or error messages. In contrast, the learnability- and satisfaction-driven versions implement input validation, accepting only valid inputs. Location inputs are restricted to decimal values within accepted ranges. For file inputs, a dedicated section displays the parsed information, clarifying what the program has actually processed. The rationale behind this difference lies in finding the optimal position along the expert-novice user spectrum. While expert users may already be familiar with valid input ranges, novice users may not be as familiar and would subsequently benefit from more UI feedback.

3.6 Design & Implementation Trade-offs

This section discusses the design and implementation decisions made during development and their impact on properties such as readability, maintainability, and coupling. Specifically, it will discuss how these decisions influenced the application's overall structure and behavior.

The primary distinction between the efficiency- and learnability-driven designs was the shift from a single view containing all information to segmented views, akin to flipping through a manual. A similar issue existed on the backend. In the efficiency-driven design, our controller component, the *SettingsManager* class, handled some responsibilities of the model component. In practice, the model classes primarily stored data, while most computation occurred in the controller, resulting in highly coupled code. In the learnability-driven design, more computation was assigned to individual model classes, and the controller primarily orchestrated the model, view, and application, serving as a middleman. Although this adjustment did not reduce coupling, it improved readability and maintainability.

A key design decision concerned memory management and inter-class communications. Initially, classes were allocated on the stack to allow for simple communication. But this caused the program to exceed the default stack frame size, leading to a crash. To address this issue, two actions were proposed: first, a CMake command to increase the stack frame size; second, to declare classes on the heap. It was decided to use heap allocation instead of defining custom compiler behavior. A drawback to this decision was increased complexity in inter-class communications. To mitigate this increased complexity, signals and slots were utilized. This change also significantly reduced coupling, as classes no longer needed to rely on other classes' header files and function declarations.

The introduction of signals and slots allowed for more decoupled communication between classes. When one class needed to communicate with another, it simply emitted a signal, without knowledge of the receiving class or maintaining function pointers. Similarly, receiving classes only needed to implement a generic slot for incoming communication. This turned communication into a more event-driven system, reducing coupling but increasing system complexity.

Chapter 4

Method and Result

In this chapter, we discuss our testing strategy and the results of the exploratory study. We will begin by introducing the evaluation methodology and subsequent literature. Continuing, we will address how our study deviates from standard and recommended practices and provide our justifications for these deviations. Finally, we will present the results of the testing phase.

4.1 Method

In this section, we will describe the testing methodology used. This will be followed by an explanation of how the test was implemented and executed, including the questionnaire, the participants, and the pilot study. We will then present the results of both the testing and the pilot study, followed by an analysis.

4.1.1 SUS

The SUS is intended to evaluate the usability of systems. However, since the program is likely to undergo continuous development after the conclusion of this thesis project, we have adopted an augmented evaluation method. In addition to the standard SUS questionnaire, testers will also be asked to complete an inter-design comparison and provide free-text feedback.

The designs are intentionally extreme, intended to maximize specific metrics. By supplementing the SUS with inter-design comparisons and free-text feedback, testers can provide general feedback through the SUS and highlight specific design features. This approach offers a more detailed comparison, enabling testers to explain their scores and identify concrete design features that will inform further development.

Because the intended user group is relatively small and the application will be limited rather than open to the general population, we can deviate from standard test sample sizes. Instead of recruiting enough testers to generalize to the general public, we aim

to recruit a smaller group that represents a percentage of the intended user base. This approach was necessary due to the limited pool of potential testers, which made it challenging to recruit a sample large enough for generalization.

4.1.2 Test Arrangement

In their book *Handbook of Usability Testing*, J. Rubin and D. Chisnell provide extensive guidance on planning and conducting usability tests [12]. Supplementing this with research and consultation from Professor John Sören Petterson at Karlstad University enabled us to develop and implement our testing strategy.

First, it is important to note that our definitions of *efficiency*, *learnability*, and *satisfaction* differ from those of Rubin and Chisnell. In this thesis, we use these terms as memorable labels for aspects of cognitive load, whereas Rubin and Chisnell define them as follows.

”**Efficiency** is the quickness with which the user’s goal can be accomplished accurately and completely”.

”**Learnability** is part of effectiveness and has to do with the user’s ability to operate the system to some defined level of competence after some predetermined amount and period of training...”.

”**Satisfaction** refers to the user’s perceptions, feelings, and opinions of the product...”.

Our testing strategy follows an exploratory study approach. The goal of such a study is to investigate preliminary design features rather than fully or partially developed products. Rubin and Chisnell emphasize this as a crucial step in the development process, as the results often inform critical design features.

An exploratory study typically requires extensive interaction between the participant and the test moderator, ideally in person. However, because many participants are located across Sweden, in-person testing is not feasible.

The authors strongly recommend using screen and audio recording during testing, particularly for remote sessions. However, in the sensitive environment of the defense sector, installing such software on company or government computers is usually not possible. This necessitates a more independent form of testing with less moderator involvement than usual.

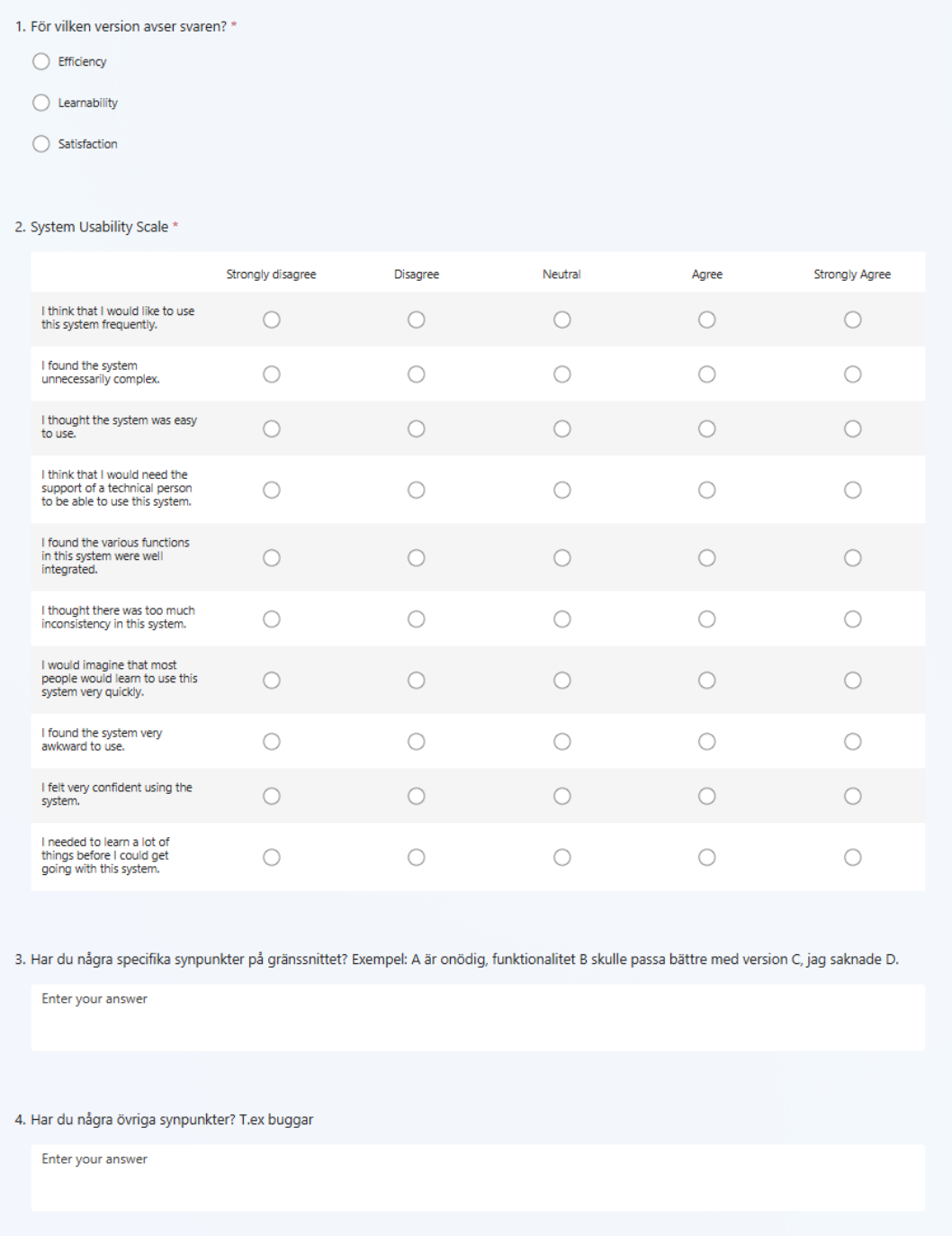
Independent remote testing raises some concerns. First, the reliability of the data is a consideration, as it is now entirely self-reported. Given that each participant will work with the application we are developing and that the testing period is relatively long (almost three weeks), we expect accurate self-reporting.

Second, remote testing limits the recommended high level of tester interaction. However, research by Professor John Sören Petterson et al. shows that remote testing offers many benefits, mainly due to fewer distractions from inexperienced

moderators [11]. Since the application will be developed iteratively and in close collaboration with the intended user group, we hope this approach will mitigate potential limitations.

Latin squares were used to randomize the order in which each test subject evaluated the applications, as recommended by Sören Petterson, to prevent test order from becoming a confounding variable. Testers received a set of optional tasks to help them explore each application, and they were informed they could spend as much or as little time testing as they wished. Because screen and audio recording were not feasible, we did not require mandatory tasks to record timing or observe actions.

After testing each version, participants completed the first of two forms, both of which were created using Microsoft Forms. This form (see Figure 4.1.1) began with a section for the tester to indicate which application they were evaluating. This was followed by a Likert scale for the SUS score and two optional free-text sections for qualitative feedback.



1. För vilken version avser svaren? *

- ◯ Efficiency
- ◯ Learnability
- ◯ Satisfaction

2. System Usability Scale *

	Strongly disagree	Disagree	Neutral	Agree	Strongly Agree
I think that I would like to use this system frequently.	◯	◯	◯	◯	◯
I found the system unnecessarily complex.	◯	◯	◯	◯	◯
I thought the system was easy to use.	◯	◯	◯	◯	◯
I think that I would need the support of a technical person to be able to use this system.	◯	◯	◯	◯	◯
I found the various functions in this system were well integrated.	◯	◯	◯	◯	◯
I thought there was too much inconsistency in this system.	◯	◯	◯	◯	◯
I would imagine that most people would learn to use this system very quickly.	◯	◯	◯	◯	◯
I found the system very awkward to use.	◯	◯	◯	◯	◯
I felt very confident using the system.	◯	◯	◯	◯	◯
I needed to learn a lot of things before I could get going with this system.	◯	◯	◯	◯	◯

3. Har du några specifika synpunkter på gränssnittet? Exempel: A är onödig, funktionalitet B skulle passa bättre med version C, jag saknade D.

Enter your answer

4. Har du några övriga synpunkter? T.ex buggar

Enter your answer

Figure 4.1.1: The first evaluation form

The second form (see Figure 4.1.2) was designed for inter-design comparison. Testers completed this form after evaluating all three applications. The form consisted of three sections: a ranking of the applications by preference, an optional field to provide an email address for thesis notification, and an optional section for additional comments.



Figure 4.1.2: The second evaluation form

4.1.3 Test Participants

One month before testing began, 11 individuals were identified as potentially interested in participating, representing a majority of users currently working with the GBM (approximately 15–20 individuals). These individuals included BAE employees as well as representatives from the military and several government agencies. After being contacted, one individual declined to participate, while another was recommended by an existing participant. During testing, issues with scheduling and accessing the testing material resulted in some testers being unable to participate. After a second round of recruitment, a total of 11 testers participated.

These individuals brought diverse backgrounds, including engineering, research, and teaching. Involving participants from such a broad range of occupations early in development provides varied perspectives, helps prevent software ossification, and ensures input from multiple viewpoints.

Although participants come from various occupations, the sample size is relatively small and unevenly distributed, with engineers comprising more than half. This introduces a risk of bias, where the results may overrepresent engineering perspectives relative to research and teaching. While the study focuses on GUI and layout design, usability perceptions may still vary across user groups due to differences in experience, work tasks, and familiarity with similar systems. The results should therefore be interpreted with caution.

Given the exploratory nature of this study and the relatively small intended user group, the sample is still considered representative of a portion of the target population. Continued engagement with a broader and more balanced set of users in future iterations is expected to reduce this limitation and improve the generalizability of the findings.

4.1.4 Pilot Study

As recommended by Rubin and Chisnell, a pilot study was conducted one week before the main testing phase. In this pilot study, a tester who was not part of the main testing group evaluated the three applications, completed the assigned tasks, and completed the two questionnaires. The pilot study had two main goals: primarily, it served as quality control, helping to identify major issues before the main testing phase began; secondarily, it allowed us to record the time required to complete the tasks.

To investigate whether completion times decreased over time, the tester repeated the testing two weeks later. While a sample size of one does not yield statistically significant results, it can suggest a trend and help draw qualitative conclusions.

The tester performed the same optional tasks assigned to the main testing group, and completion times were recorded for both sessions. Initially, two testers were recommended to capture any difference between remote and *on-site* testing with a moderator present. However, since it became clear that all testing would be conducted remotely, the pilot study was reduced to a single tester.

4.1.5 Test Execution

Five days before the test period began, participants received an email reminding them about the upcoming testing. The email also clarified some practical aspects of the process. Specifically, it began with a brief introduction to the three designs, the rationale for conducting an exploratory study, and instructions for conducting the testing.

The email then reiterated the test period's duration, which was nearly three weeks. This timeframe was chosen to give test participants ample time to evaluate the applications without stress or the need to rearrange their schedules. The first week of testing coincided with the Easter holidays, reducing the total period by two business days. Finally, the email informed testers about the use of Latin squares, noted that the

applications were still early in development and might contain bugs, and included a list of known bugs.

It was decided not to review the results until after the testing phase deadline. Since the testing order was known, premature analysis of the results could have revealed the identities of one or more test subjects, which would be unacceptable.

4.2 Results

This section presents the results of the testing phase. First, we present the quantitative data, beginning with the SUS scores and followed by the inter-design comparison. Next, we present the qualitative free-text feedback. Finally, we report the completion times recorded during the pilot test. The raw SUS scores and pilot test completion times are provided in Appendix .7. The results presented in this section, and the analysis in the following section can not be extrapolated into other projects or contexts. The number of test participants fall below what is required to generalize the wider population. However, the number of test participants is large enough to generalize of the intended user group. Despite this, the mean, interquartile range, and standard deviation are given as to provide context for the data in figure 4.2.1. Statistics on each individual SUS question is also provided in Appendix .7.1.

The testing included 11 participants, resulting in 33 responses to the first form and 10 responses to the second form. For the first form, across all versions, the SUS score had a mean of 59.3, a mean interquartile range of 20.8, and a mean standard deviation of 14.1. The efficiency-driven design produced a mean SUS of 68.5, an interquartile range of 18.8, and a standard deviation of 11.5. The learnability-driven design produced a mean SUS of 45.7, an interquartile range of 21.3, and a standard deviation of 17.5. The satisfaction-driven design yielded a mean SUS score of 63.9, an interquartile range of 22.5, and a standard deviation of 13.2.

Design	Mean	Interquartile Range	Standard Deviation
Efficiency-driven	68.5	18.8	11.5
Learnability-driven	45.7	21.3	17.5
Satisfaction-driven	63.9	22.5	13.2
Mean	59.34	20.83	14.05

Table 4.2.1: SUS score statistics

The second form asked testers to rank the designs from best to worst based on which design they preferred. The purpose of this form was extract more discrete results in case the SUS scores turned out to be very similar. The efficiency-driven design placed first six times, second three times, and third once. The learnability-driven design never placed first, placed second three times, and third seven times times. And finally, the satisfaction-driven design placed first four times, second four times, and third twice (see figure 4.2.2).

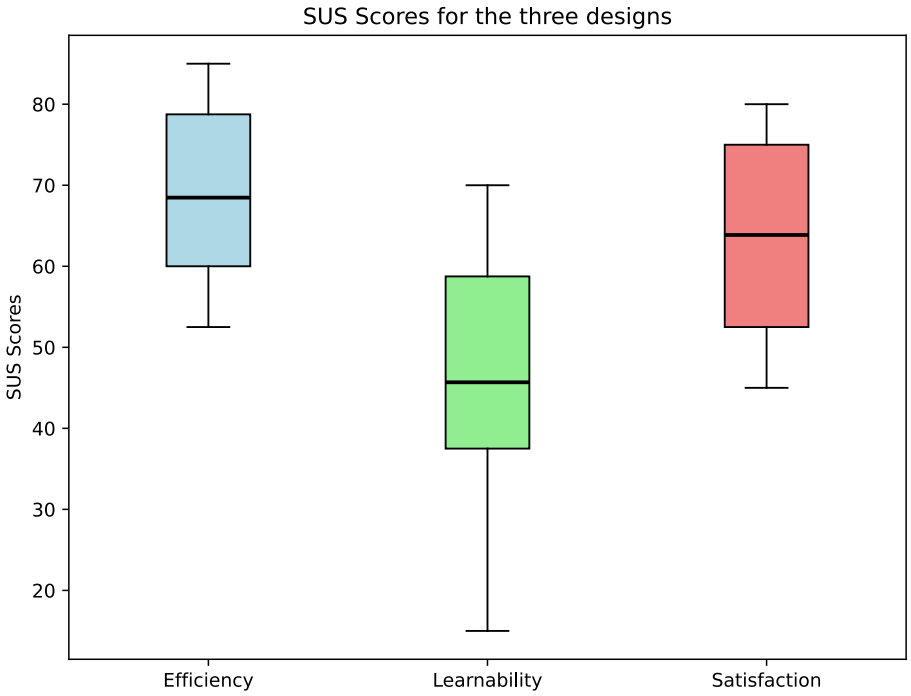


Figure 4.2.1: SUS scores for the three versions. Higher is better.

A total of 47 free-text responses were provided across both forms, 42 responses in the first form, and five responses for the second form.

Completion times of the example tasks were measured at two sessions, two weeks apart. The results are summarized in table 4.2.2, and visualized in figure 4.2.3 and 4.2.4. The first session of the pilot study yielded in 371 seconds (4:11) for the efficiency-driven design, 698 seconds (13:38) for the learnability-driven design, and 595 seconds (9:55) for the satisfaction-driven design. The second session yielded in 226 seconds (3:46) for the efficiency-driven design, 530 seconds (8:50) for the learnability-driven design, and 319 seconds (5:19) for the satisfaction-driven design.

From the first to the second session, completion time for the efficiency-driven design was reduced by 39.1% (2:25), the learnability-driven design by 24.1% (2:48), and the satisfaction-driven design by 46.4% (4:36). This results in a mean reduction of 39.1%.

Design	Session #1	Session #2	Reduction
Efficiency-driven	4:11	3:46	-39.1%
Learnability-driven	13:38	8:50	-24.1%
Satisfaction-driven	9:55	5:19	-46.4%

Table 4.2.2: Pilot study overview

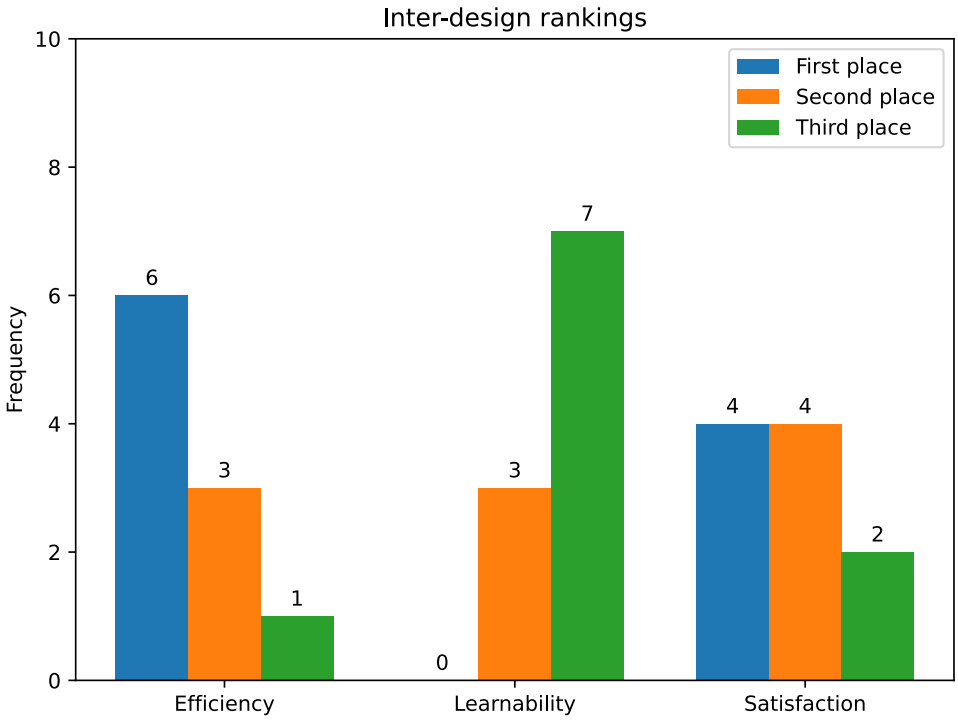


Figure 4.2.2: Inter-design comparison.

4.3 Analysis

With only 11 test participants, no statistically significant conclusions can be drawn from the data; the same applies to the pilot study completion times. However, Nielsen writes that, regarding usability testing, the vast majority of conclusions should come from qualitative research [24]. Furthermore, following Professor John Sören Pettersson's recommendation, the quantitative results, i.e., the SUS scores and pilot study completion times, should be used solely as descriptive statistics, rather than to characterize the entire user group.

Comparing SUS scores across the three versions does not allow for any definitive conclusions, given the data overlaps and the small sample size. Applying a letter grading score to the mean SUS scores for each design (with 60 being considered a passing score), both the efficiency- and satisfaction-driven designs receive passing scores, while the learnability-driven design scores below passing. However, it is important to note that a letter grading scale is an inprecise quality indicator given that the SUS can not be normalized.

With a consistent reduction in completion times and a mean reduction of 39%, the data suggest that extended use of all three designs results in shorter task completion times. The efficiency-driven designs produced the lowest task completion times in both sessions, aligning with Rubin and Chisnell's definition of efficiency, and indicating that the efficiency-driven design is the most effective.

The satisfaction-driven design saw the greatest reduction in completion time. This might suggest that the satisfaction-driven design is better for learnability than

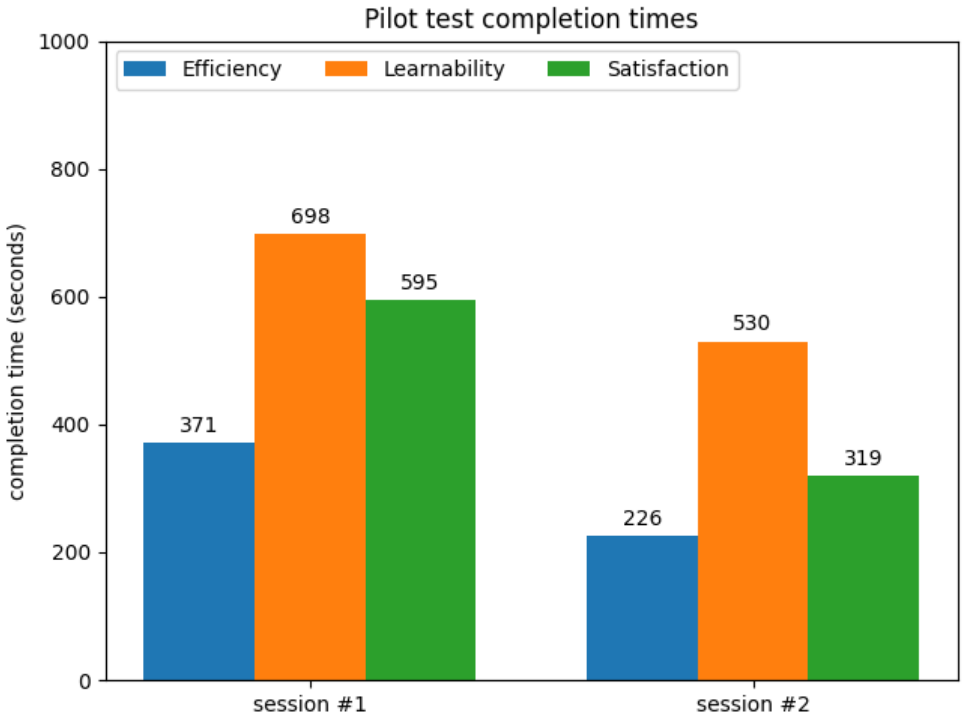


Figure 4.2.3: Completion times. Lower is better.

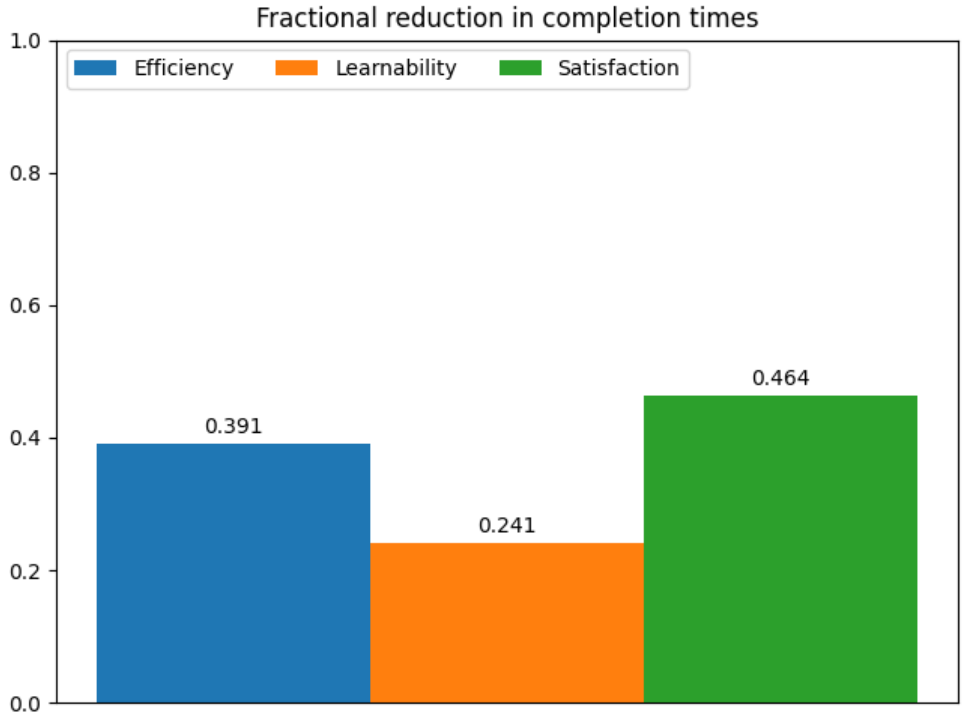


Figure 4.2.4: Fractional reduction. Higher is better.

the learnability-driven design, according to Rubin and Chisnell's definition of learnability.

Nearly all participants completed the second form, and from it we can draw fairly confident conclusions, despite the low participation rate. The learnability-driven design was ranked worst with a frequency three, and six times the other designs respectively. Furthermore, the learnability-driven design was never ranked first place. Contrast this to the efficiency- and satisfaction-driven designs that ranked relatively similar. This could indicate that the optimal UX design is a combination of design-features form the efficiency- and satisfaction-driven designs. It is also safe to say that the learnability-driven design is not satisfactory given its SUS scores and low ranking.

Around 25% of the free-text feedback concerned bugs. Most bugs were known prior to testing, but had not been resolved due to time constraints. It is important not to dismiss this feedback as a byproduct of the designs being prototypes, and instead use it to highlight issues and deficiencies in the development process.

From the free-text responses not related to bugs, 26 aggregated points were identified across the three designs. Out of these, seven can be generalized across all designs:

1. Allow the window to be resized
2. Synchronize naming conventions to existing weapon-dependent OPr's
3. Implement keyboard shortcuts
4. Display features of selections (weapon, shell, fuze etc.)
5. Support configuration files for default values
6. Display alternative selections
7. Ability to export calculations

Points one through four reflect relatively simple changes that can be easily implemented. Keyboard shortcuts are already partially supported, allowing users to use the Tab key to navigate input fields. Configuration files for default values, where common configurations can be automatically entered, could help automate repetitive work. Displaying alternative selection combinations in which the same target position is reached with different settings is a supported GBM feature and was initially planned to be supported. However, this feature was not implemented due to time constraints. Exporting calculation results is like points one through four relatively simple to implement. It might also be useful to investigate existing data format across NATO to further facilitate data sharing.

In the following sections, we will analyze the design-specific feedback. This feedback will be used in Chapter 5 to draw conclusions about which design features are beneficial/detrimental to perceived usability.

Efficiency

Testers generally described the efficiency-driven design as fast and easy to use, especially compared to the other designs. This feedback was mostly attributed to the design being monoview and to the fact that users did not have to navigate multiple views to make calculations.

Some testers requested that the design be segmented to better highlight which elements were related to one another. Specifically, testers suggested frames, colored backgrounds, and vertical lines. These are good suggestions for a prototype context, but depending on further development, such separation techniques might not be sufficient, and multiple views might need to be investigated.

Several testers requested clearer visual feedback to indicate valid input ranges and whether files were loaded. In contrast to the efficiency-driven solution that has little to no feedback, both the learnability- and satisfaction-driven designs use free-text input fields, which are restricted in what values can be entered, and the input range is clearly displayed in the immediate vicinity of the input field. The learnability- and satisfaction-driven designs also both display which files are loaded, or not loaded.

Two testers expressed a desire for fields related to trajectory calculations to either be moved from the text file and given their own input fields or removed completely. These comments align with the results of brainstorming for potential future development.

Learnability

The learnability-driven design achieved the lowest SUS score, as reflected in the free-text feedback. Testers generally described feeling ”restricted” and ”locked” by the checklist. They also described frustration with having to navigate many views each time they wanted to run a new calculation.

A common reason testers felt restricted was the order of elements in the checklist and the need to traverse the entire checklist for every single calculation. The design does not require users to traverse every element in the checklist, only the ones handling mandatory inputs. But evidently, this was not being communicated to the user clearly enough. A related issue was that adjusting calculations was cumbersome, as the entire checklist would need to be re-traversed.

Several testers expressed frustration that the only way to navigate the checklist was with the two arrow buttons, and that they could not jump to a specific view by clicking the corresponding element in the checklist.

Many testers thought the usage of button icons was bad and should not be used. This was an expected response and was included only to verify that this was the testers’ opinion.

However, several testers reported that the design made it easy for them to build a

mental model and ”see the full picture”.

Satisfaction

Testers generally described the satisfaction-driven design as being a good split between the efficiency- and learnability-driven designs. Feedback specifically noted that the input fields in the *Positions*, *Weapons*, and *Files* views were all interrelated.

Two points were given related to increasing the clarity of the results. The first point requested that input parameters should be visible in the results view to better show what parameters a given result is based on. The second point asked that a settings panel be visible at all times. In both the efficiency- and learnability-driven designs, unit controls were always visible to the user, whereas in the satisfaction-driven design, unit control resides in the *Settings* view.

The *Files* view was subject to two suggestions. Firstly, it was suggested that the filenames of loaded files be displayed. Since the textboxes displayed a parsed version of the files, it might not be clear which file is loaded. The second suggestion asked for the ability to either unload or disable the usage of a text file.

The last two suggestions were related to the buttons on the left and right sidebars. The first point noted that some of the button labels were ambiguous. The second point noted that buttons should utilize different colors or shapes to indicate meaning.

Chapter 5

Conclusions and Future Work

In this chapter, we discuss the results presented in Chapter 4, reflect on practical lessons learned during the project, and analyze their effects. Finally, we present potential future work from both academic and practical development perspectives.

5.1 Conclusion

In this thesis, we aimed to investigate which design features yielded the highest perceived usability for a specialized, safety-critical tool. To isolate a single variable, cognitive load theory was used to create three design prototypes, each representing a split between intrinsic and extraneous cognitive load. A majority of current users from the intended user group were then allowed to test and evaluate the designs using both quantitative measures (SUS and inter-design rankings) and qualitative measures (free-text responses). The results allowed us to answer not only the proposed research questions but also identify several miscellaneous conclusions.

The results show that the efficiency- and satisfaction-driven GUI design philosophies yield higher perceived usability than the learnability-driven approach for the intended user group, as indicated by SUS scores and inter-design rankings (**RQ1**). Further, by combining the SUS scores, inter-design rankings, and tester feedback, the GUI design philosophy that integrates elements from both efficiency- and satisfaction-driven designs appears to be the most suitable, as it balances efficiency and user guidance (**RQ2**). This answers the proposed research questions.

By combining the quantitative SUS scores, completion times, and inter-design rankings with the qualitative free-text responses, we can produce a comprehensive summary. Although the three designs are not yet production-ready, given mean SUS scores in the 45-70 range (constituting failing to barely passing scores according to letter grading scales), they meet the requirements for this exploratory study. Each design has provided important insights for future development, but the number of bugs and limited functionality have generally undermined perceived usability. Nevertheless,

since the designs were created for an exploratory study, they fulfill their intended use cases.

Free-text responses confirm that the designs relate to both intrinsic and extraneous cognitive load. Complaints regarding the learnability-driven design mostly concern the interface providing excessive assistance, users feeling restricted by the checklist structure, and frustration with repeatedly traversing multiple views for calculations. In contrast, most complaints about the efficiency-driven design point to a lack of user feedback and features for efficient calculation.

Additionally, several design features have been identified as positively impacting perceived usability. Tester feedback particularly highlighted that input fields clearly indicating valid input ranges, visible settings, and showing which resources and files are currently loaded improve usability and user confidence. Conversely, forcing users to follow a checklist or predetermined input sequence negatively affects perceived usability, particularly for experienced users who wish to adjust calculations quickly.

Design features should be segmented into views to the smallest practical extent. Each segment should separate design features by type (e.g., optional resources or projectile properties) or chronologically (e.g., input- and output-related features).

Finally, features that promote clear, unambiguous data import and export have been shown to improve usability. Examples include displaying filenames and contents of loaded resources, as well as enabling export to commonly used data formats.

Given the small number of test participants, it is not possible to draw statistically significant conclusions that generalize to the broader population. However, since the participants represent most individuals working with the GBM, we argue that conclusions can still be generalized to the intended user group.

The conclusions of this thesis will form the basis for discussions and planning of a new application that serves as a highly usable tool for visualizing GBM ballistic calculations. The established tester group can continue to provide feedback on new features as development progresses.

5.2 Discussion

SUS scores and rankings indicate that the efficiency- and satisfaction-driven designs outperform the learnability-driven design. However, this does not necessarily imply that features that reduce extrinsic cognitive load always result in higher usability than those that target extraneous load. Instead, testers favored a balance between speed and guidance, structure and flexibility, and control and support within the context of the limited prototypes.

The qualitative feedback further supports this interpretation. Testers frequently described the learnability-driven design as restrictive due to its checklist-based

workflow and the need to repeatedly navigate between multiple views. At the same time, several testers appreciated how the segmented structure helped them build a mental model and better understand the calculation process. In contrast, the efficiency-driven design was generally perceived as faster and more flexible, but also lacking sufficient user feedback and guidance. The satisfaction-driven design appears to have achieved a more balanced trade-off between these competing properties.

Since only one round of testing was conducted, there is a risk of the *novelty effect* acting as a confounding variable. The novelty effect occurs when initial test scores are elevated because testers are exposed to something new, resulting in increased dopamine release in the brain [1]. This could artificially raise SUS scores if used as the primary metric. And since our primary goal was to compare the designs against each other using Latin squares, the novelty effect should have minimal impact on our conclusions. However, there is still a risk of user preferences changing as the system evolves, particulary in regards to additional features, changes in user group composition, and the maturation of the GBM, all of which could influence user expectations and interaction patterns.

During the testing phase, multiple reminders and, in some cases, extended deadlines were necessary to obtain responses from testers. Several factors may have contributed to this. One likely reason is the high degree of tester independence. Because testing was conducted remotely and testers could choose their testing times, some may have procrastinated in favor of their regular work duties. Additionally, the nearly one-month gap between initial recruitment and the start of testing may have caused testers to schedule time initially, only for their calendars to fill up before testing began.

A possible solution would have been to schedule testing sessions for all participants, either in person or remotely. Scheduled sessions could have improved planning and eliminated the need for extensions. However, this approach might have reduced the tester pool, as it requires a greater commitment and more upfront effort from participants, potentially lowering recruitment numbers.

Another identifiable cause of delay was that representatives from government agencies were sometimes unable to launch the applications due to IT security policies. While some issues could be resolved, others prevented testers from participating and required replacement through a second recruitment round.

5.3 Future Work

Looking ahead, it makes sense to focus not only on improving the current prototypes but also on broadening our evaluation of usability in ballistic visualization systems. These applications were built for exploratory purposes, so it's not surprising that there are still rough edges, such as stability issues, missing features, and limitations in the thoroughness of their evaluation.

From a technical perspective, the next steps should focus on making the applications

more robust, fixing bugs, adding features that were delayed due to time constraints, and generally improving the user experience. Testers noted the need for clearer feedback, stronger input validation, and more options for importing and exporting calculations. And on top of that, features like integrated map support or more advanced trajectory visualizations could really take usability and operational insight to the next level.

This thesis sets the stage for future research in many directions. Up to this point, the focus has been on a single usability metric, examined through the lens of cognitive load, and based on feedback from a relatively small group of users at a single moment in time. That leaves plenty of room to broaden both what gets measured and how deeply we look into it.

For example, longitudinal studies of users would help us see how their perceptions of usability change as the system and their experience with it evolve. This could reveal whether the trade-offs among efficiency, learnability, and satisfaction persist or fade with familiarity, and would also help us determine whether early results were skewed by the novelty of the prototypes.

It'd also be worth bringing in more concrete performance metrics, like how long tasks take or how often errors occur, to balance out the subjective ratings from tools like the SUS. Expanding the tester pool and grouping users by background or experience would give a much richer picture of how different people use and perceive the system.

Lastly, it's worth noting that research by Maqbool and Herold shows that some software quality attributes receive less attention than others [20]. Taking a closer look at aspects such as Operability and Memorability, especially those that are less often studied, could yield useful insights not just for ballistic visualization tools but for safety-critical systems in general.

Bibliography

[1] A. Lorents M.F.L. Ruitenberg, J. Schomaker. *Novelty-induced memory boosts in humans: The when and how*. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC10023963/ (visited on 05/04/2026).

[2] Allemann, L. Sakerman M. Remmet R. Rotšan K. and Reispass, K. "Comparative Study Into Archaic Software Integration Based on NATO S4 Software Library". In: 2024, pp. 80–103.

[3] BAE Systems. *What we do*. URL: https://www.baesystems.com/en/what-we-do (visited on 02/02/2026).

[4] Brooke, John. *Usability Evaluation In Industry*. Taylor Francis Group, 1996.

[5] Dargo, Sandor. *C++23: Growing unicode support*. URL: https://www.sandordargo.com/blog/2023/11/29/cpp23-unicode-support (visited on 03/02/2026).

[6] Dickinson, E. R. *The Production of Firing Tables for Cannon Artilery*. Report no. 1371. U. S. Army Materiel Command, Nov. 1967.

[7] Fessenden, Therese. *Obtaining Consent for User Research*. URL: https://www.nngroup.com/articles/informed-consent/ (visited on 04/09/2026).

[8] Hart, Sandra G. "Nasa-Task Load Index (NASA-TLX); 20 Years Later". In: *Proceddings of the Human Factors and Ergonomics Society Annual Meeting, Volume 50, Issue 9* (2006).

[9] Hunt, J W and Huntington, G N. "A comprehensive environmental investigation of an active artillery range". In: 2007, pp. 43–53. DOI: 10.1002/ffej.3330090206.

[10] J Fonor. *Development of firing tables for accuracy stretches back more than 100 years*. 2018. URL: https://www.army.mil/article/215516/development_of_firing_tables_for_accuracy_stretches_back_more_than_100_years (visited on 02/02/2026).

[11] J. Sören Petterson G. Olsson, H. Andersson. *Usability Testing of Interactive Systems Conducted by Usiversity Students During a Pandemic*. URL: https://aisel.aisnet.org/siged2022/10 (visited on 03/23/2026).

[12] J.Rubin, D. Chisnell. *Handbook of Usability Testing*. Wiley, 2nd edition, 2008.

[13] James L. Cogan, Patrick A. Haines. *METCM-based Messages from Climatological Data*. Report. Army Research Laboratory, Mar. 2013.

[14] Kaplan, Kate. *10 Usability Heuristics Applied to Complex Applications*. URL: https://www.nngroup.com/articles/usability-heuristics-complex-applications/ (visited on 04/07/2026).

[15] Lewis, J R. "The System Usability Scale: Past, Present, and Future". In: 2018, pp. 577–590. DOI: 10.1080/10447318.2018.1455307.

[16] *LIGHT MACHINE GUN - M249 Series*. Manual TC 3-22.249. U. S. Army Headquarters, May 2017.

[17] Lutkevich, Ben. *dynamic link library (dll)*. URL: https://www.techtarget.com/searchwindowsserver/definition/dynamic-link-library-DLL (visited on 04/07/2026).

[18] M- Schrepp J. Kollmorgen, J. Thomaschewski. "A Comparison of SUS, UMUX-LITE and UEQ-S". In: *Journal of User Experience, vol. 18, Issue 2* (2023).

[19] Mahlke, Sascha. "User Experience of Interaction with Technical Systems". PhD thesis. University of Berlin, 2008.

[20] Maqbool, Bilal and Herold, Sebastian. "Potential effectiveness and efficiency issues in usability evaluation within digital health: A systematic literature review". In: *Journal of Systems and Software* 208 (2024), p. 111881. ISSN: 0164-1212. DOI: https://doi.org/10.1016/j.jss.2023.111881. URL: https://www.sciencedirect.com/science/article/pii/S0164121223002765.

[21] Mozilla. *MVC*. URL: https://developer.mozilla.org/en-US/docs/Glossary/MVC (visited on 04/22/2026).

[22] NASA. *NASA Task Load Index (TLX)*. URL: https://www.nasa.gov/human-systems-integration-division/nasa-task-load-index-tlx/ (visited on 03/30/2026).

[23] Nielsen, Jakob. *10 Usability Heuristics for User Interface Design*. URL: https://www.nngroup.com/articles/ten-usability-heuristics/ (visited on 04/07/2026).

[24] Nielsen, Jakob. *How Many Test Users in a Usability Study?* URL: https://www.nngroup.com/articles/how-many-test-users/ (visited on 04/08/2026).

[25] Nielsen, Jakob. *Novice vs. Expert Users*. URL: https://www.nngroup.com/articles/novice-vs-expert-users/ (visited on 03/24/2026).

[26] P. Savioja M. Liinasuo, H. Koskinen. "User experience: does it mater in complex systems?" In: *Springer* (2013).

[27] Qt. *Aobut Qt*. URL: https://wiki.qt.io/About_Qt (visited on 02/02/2026).

[28] Qt. *QML Syntax Basics*. URL: https://doc.qt.io/qt-6/qtqml-syntax-basics.html (visited on 02/27/2026).

[29] Qt. *Qt Overviews*. URL: https://doc.qt.io/qt-6/overviews-main.html (visited on 03/02/2026).

[30] Qt. *Signals & Slots*. URL: https://doc.qt.io/qt-6/signalsandslots.html (visited on 02/02/2026).

[31] Qt. *The Meta-Object System*. URL: https://doc.qt.io/qt-6/metaobjects.html (visited on 02/02/2026).

[32] Rafifing, Neo, Mphale, Ofaletse, and Asare, Sampson D. "Exploring User perceptions of an E-Government System in Botswana Using System Usability Scale Model". In: *2022 IEEE 13th International Conference on Software Engineering and Service Science (ICSESS)*. 2022, pp. 195–198. DOI: 10.1109/ICSESS54813.2022.9930194.

[33] *REGULATION (EU) 2016/679 OF THE EUROPEAN PARLIAMENT AND OF THE COUNSIL*. 2016.

[34] *Software Design for Maintainability, NASA DFE-6*. 1994.

[35] Starbrink, N. V. "EFFICIENCY BY DESIGN". In: *Umeå University* (2023).

[36] Sweller, John. "CHAPTER TWO - Cognitive Load Theory". In: ed. by Jose P. Mestre and Brian H. Ross. Vol. 55. Psychology of Learning and Motivation. Academic Press, 2011, pp. 37–76. DOI: https://doi.org/10.1016/B978-0-12-387691-1.00002-8. URL: https://www.sciencedirect.com/science/article/pii/B9780123876911000028.

[37] *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. 2011.

[38] Tipp, Andrew. *Usability testing: the complete guide*. URL: https://uxplanet.org/usability-testing-the-complete-guide-e162898f68db (visited on 04/01/2026).

Appendix - Contents

.1 Building and Initialization 50
.2 GCS coordinates 51
.3 Custom Weather file 51
.4 Fire limits 52
.5 Crests 53
.6 Trajectory 53
.7 SUS Scores 54
.7.1 Statistics on individual SUS questions 55
.8 Pilot Test Completion Times 56
.9 Full-sized figures 56

.1 Building and Initialization

We begin by creating a CMake file to build our application. This file has three main responsibilities: initializing the Qt MOS, linking with the GBM, and adding a post-build step to move our dependencies after building.

```
cmake_minimum_required(VERSION 3.16)
project(Vera VERSION 0.1 LANGUAGES CXX)
qt_standard_project_setup(REQUIRES 6.10.1)

set(GBM_LIB_DIR ${CMAKE_CURRENT_SOURCE_DIR}/Vendor/gbm/libs)
set(DLL_FILE libgbm.dll)

qt_add_executable(appVera src/main.cpp)
qt_add_qml_module(appVera
    URI Vera
    SOURCES <source files>
    QML_FILES <qml files>)

add_library(GBM_lib SHARED IMPORTED GLOBAL)

add_custom_command(TARGET appVera PRE_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy ${GBM_LIB_DIR}/${DLL_FILE} ${
   CMAKE_BINARY_DIR}/${DLL_FILE})

add_custom_command(TARGET appVera POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy ${GBM_LIB_DIR}/${DLL_FILE} ${
   CMAKE_CURRENT_SOURCE_DIR}/build/${CMAKE_BUILD_TYPE}/${DLL_FILE})
```

Listing 1: Shortened CMake build file

After making our CMake file we continue to our applications entry point. In our main function we first initialize the GBM with *GBM_Begin()*. Following this we initialize and link both the Qt Application and the Qt engine using the *connect()* function. Finally we expose the rest of our code to the engine and MOS.

The *SettingsManager* class acts as our MVC controller component. The controller then initializes 11 different classes on its stack, acting as our model component. It then waits to be called on by the QML frontend.

```
int main(int argc, char* argv[]) {
    GBM_Begin();

    QGuiApplication app(argc, argv);
    QQmlApplicationEngine engine;
    QObject::connect(&engine, &QQmlApplicationEngine::objectCreationFailed,
                     &app,    []() {QCoreApplication::exit(-1);},
                     Qt::QueuedConnection);

    engine.loadFromModule("Vera", "Main");
    qmlRegisterSingleTonType<SettingsManager>("Vera", 1, 0, "
   SettingsManager", [](QQmlEngine *, QJSEngine *) -> QObject* {
        return new SettingsManager();
```

```
    });

    app.exec();

    GBM_End();
    return 0;
}
```

Listing 2: Application entry point

.2 GCS coordinates

```
struct geoPosition {
    double latitude;
    double longitude;
    double altitude;
};
```

Listing 3: Storing a GCS coordinate

.3 Custom Weather file

The GBM has native support for three types of atmospheres: the ICAO standard atmosphere, the armed forces equivalent standard atmosphere, and the custom atmosphere METCM. ICAO and the armed forces' standard atmospheres can be applied by simply calling GBM functions, whereas custom atmospheres are defined in user-created text files. The format of the custom atmosphere closely follows the NATO standard. The data is comma and line-separated.

The METCM format described in NATO Standardization Agreement (STANAG) 4082 is relative to an Meterological Datum Plane (MDP). This can be thought of as a horozontal plane with the altitude of a measuring station. Figure .3.1 showcases the standard datum plane, which is used in aeronautical navigation. The MDP is similar to the standard datum plane, but its altitude is set to the measuring station's altitude. An example custom METCM file can see seen in appendix

```
0                                   //Validity in minutes (0=never expires)
68                                  //Lattitude of MDP
28                                  //Longitude of MDP
100                                 //Altitude of MDP
900                                 //MDP air pressure in Pa
true, 4.7, 20.0, 297.7, 97200.0     //Opitonal 0..31 altitude zones
...
```

Listing 4: Example of a custom METCM file

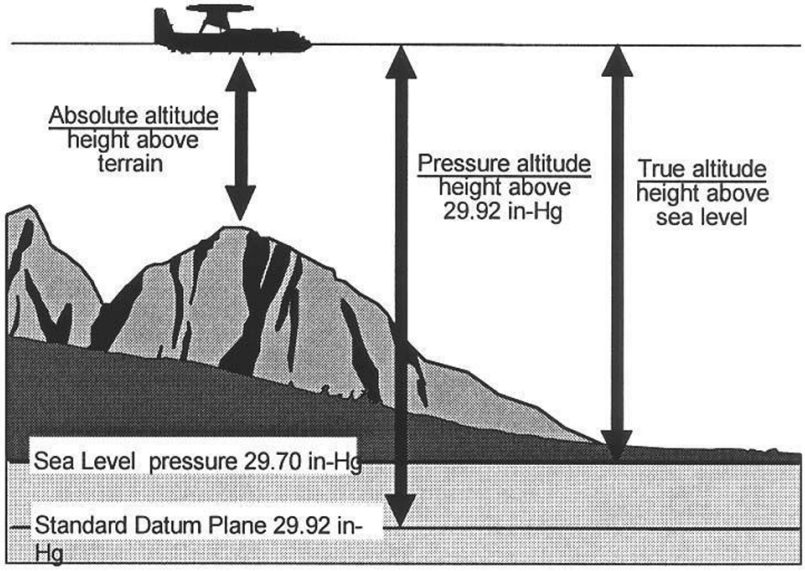


Figure .3.1: Datum Plane Visualization. Source US Navy

.4 Fire limits

Like weather, the GBM supports three types of fire limits: SFL, NFA, and FOF, along with a maximum altitude limit. NFAs and FOFs have the ability to take the form of either an ellipse or as free GCS points. This means we only need to support three formats: SFLs, ellipses, and free GCS points, hereafter referred to as polygons.

The format for fire limits is based on labels. Each fire limit starts and ends with a label indicating its type. This allows for multiple fire limits to live in the same file. The exception here is the maximum altitude restriction. Since it only requires one line, only one label is used.

```
SFL_BEGIN
59.3349, 14.6414, 101       //SFL center
2                           //center radius in meters
3                           //minimum range in meters
4                           //maximum range in meters
5                           //left boundary in radians
6                           //right boundary in radians
30                          //minimum charge
65                          //maximum charge
Projectile1                 //projectile name
Charge5                     //charge name
FuzeMode3                   //fuze mode name
100000                      //start time in Unix time
100000                      //end time in Unix time
SFL_END

NFA_POLYGON_BEGIN
59.3349 ,  14.6414 ,  101   //first GCS point
59.3349 ,  14.6414 ,  102   //second GCS point
```

```
59.3349 ,  14.6414 ,  103   //third GCS point
123                         //start time in Unix time
456                         //end time in Unix time
59.3349 ,  14.6414 ,  101   //additional optional GCS points
...
NFA_POLYGON_END
```

Listing 5: Example of how fire limits can be formatted

.5 Crests

Like fire limits, crests are also based around labels, since that allows for multiple crests in the same text file. Near crests indicate obstacles close to the weapon are defined as one crest per line. Far crests, indicating obstacles along the flight path, are indicated as one crest per set of labels.

Near crests are comprised of the distance to the obstacle, the left and right boundaries, and the obstacle height. Far crests are comprised of a set of GCS points.

```
NEAR_CREST_BEGIN
20, -0.5, 0.5, 10
10, -0.3, 0.3, 15
NEAR_CREST_END

FAR_CREST_BEGIN
59.3349, 14.6314, 100
59.3599, 49.3599, 100
59.3599, 49.5, 150
FAR_CREST_END
```

Listing 6: Example of user created crests

.6 Trajectory

To perform specific trajectory calculations, quite a few new variables must be provided. These include projectile weight and initial speed, as well as barrel azimuth and elevation. In contrast to the other user files, the trajectory file format takes a key-value format without labels. There are two reasons for this. The first is that there are fewer steps involved for a user to have one file per trajectory preset than to implement a system that assigns a unique ID to each set of presets and allows the user to switch between them. The second reason is to allow testers to give feedback on which format type they prefer. The trajectory file format also supports comments, which is also a way to present different formatting options to testers.

```
# Comment
quad_elev=1
azimuth=2
motor_temp=290
```

```
proj_weight=80
v0=950
stopping_time=10
stop_altitude=0              #comment
height_of_fuze_function=0
calculate_upleg_burst=false
```

Listing 7: Example of a user created trajectory preset

.7 SUS Scores

Note: *"Tester #"* does NOT refer to a unique tester ID, but rather the chronological order of form submission.

Efficiency

Tester #	Score	Tester #	Score
1	82.5	7	55
2	75	8	65
3	65.7	9	70
4	62.5	10	82.5
5	52.5	11	85
6	57.5		

Table .7.1: Efficiency-driven SUS scores

Learnability

Tester #	Score	Tester #	Score
1	40	7	27.5
2	67.5	8	15
3	40	9	35
4	50	10	70
5	67.5	11	40
6	50		

Table .7.2: Learnability-driven SUS scores

Satisfaction

Tester #	Score	Tester #	Score
1	50	7	75
2	75	8	45
3	62.5	9	60
4	75	10	77.5
5	47.5	11	55
6	80		

Table .7.3: Satisfaction-driven SUS scores

.7.1 Statistics on individual SUS questions

Efficiency

Question #	Mean	IQR	SD
1	2.6	1.0	1.03
2	0.6	1.0	0.67
3	3.3	1.0	0.65
4	0.6	1.0	0.93
5	0.8	1.0	0.87
6	2.9	0.5	0.70
7	1.0	0.5	0.89
8	2.6	1.0	0.87
9	1.1	1.5	1.34
10	1.9	0.5	0.70

Table .7.4: SUS statistics on Efficiency-driven responses

Learnability

Question #	Mean	IQR	SD
1	1.1	1.0	0.94
2	1.7	1.5	1.27
3	2.0	2.0	1.41
4	1.3	3.0	1.56
5	2.0	1.0	1.10
6	2.2	2.0	1.47
7	2.3	2.0	1.35
8	1.7	1.5	1.27
9	1.9	1.0	1.27
10	2.0	1.0	0.89

Table .7.5: SUS statistics on Learnability-driven responses

Satisfaction

Question #	Mean	IQR	SD
1	2.3	1.0	0.91
2	1.1	0.5	1.05
3	2.8	0.5	0.87
4	2.8	1.0	0.94
5	1.1	0.5	0.87
6	2.8	0.5	0.60
7	1.4	1.0	0.92
8	2.7	1.0	0.91
9	0.8	1.0	0.75
10	2.1	1.0	0.70

Table .7.6: SUS statistics on Satisfaction-driven responses

.8 Pilot Test Completion Times

Design	Session #1	Session #2
Efficiency	371	226
Learnability	698	530
Satisfaction	595	319

Table .8.1: Completion times in seconds

.9 Full-sized figures

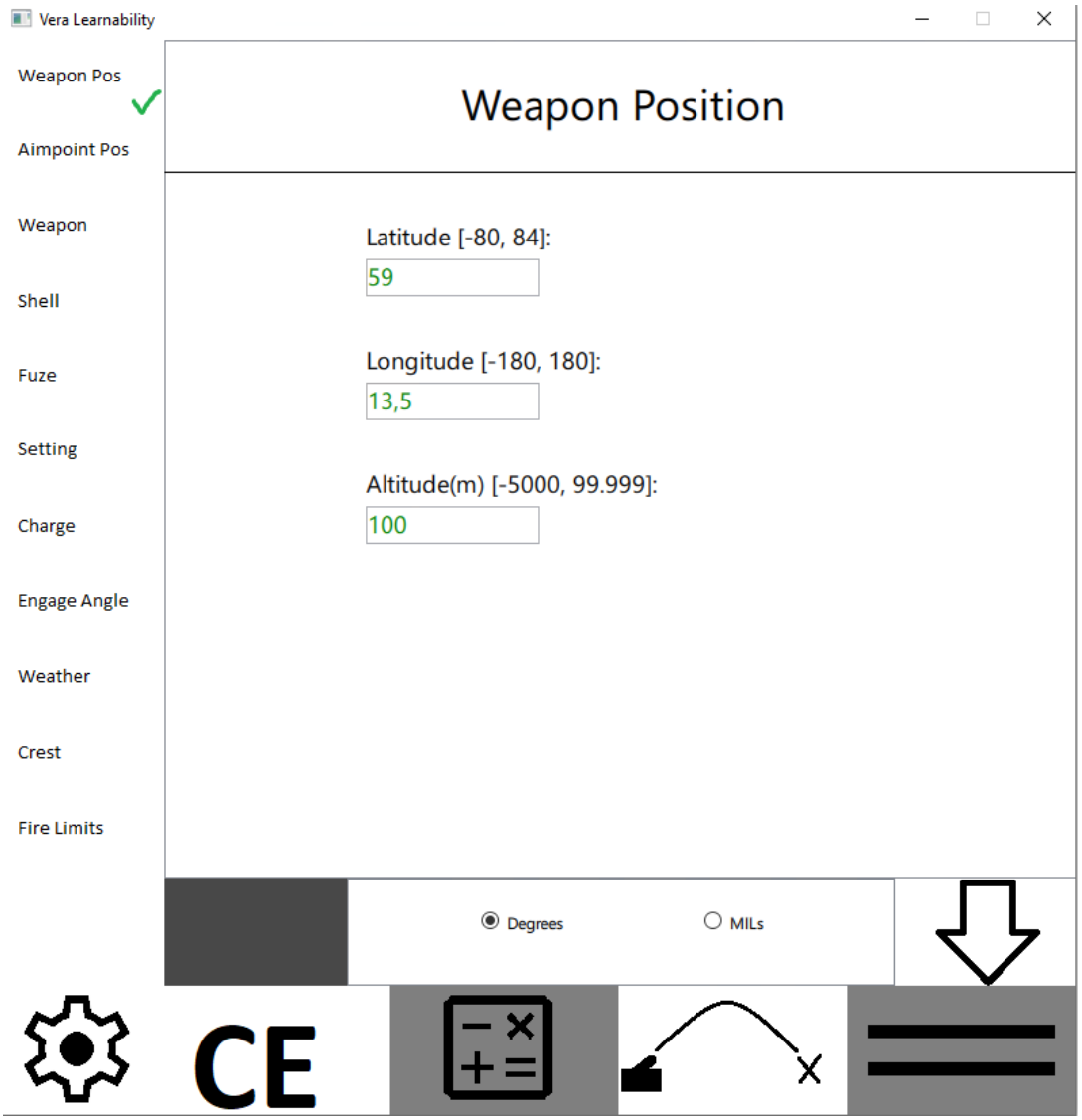


Figure .9.1: Weapon position view

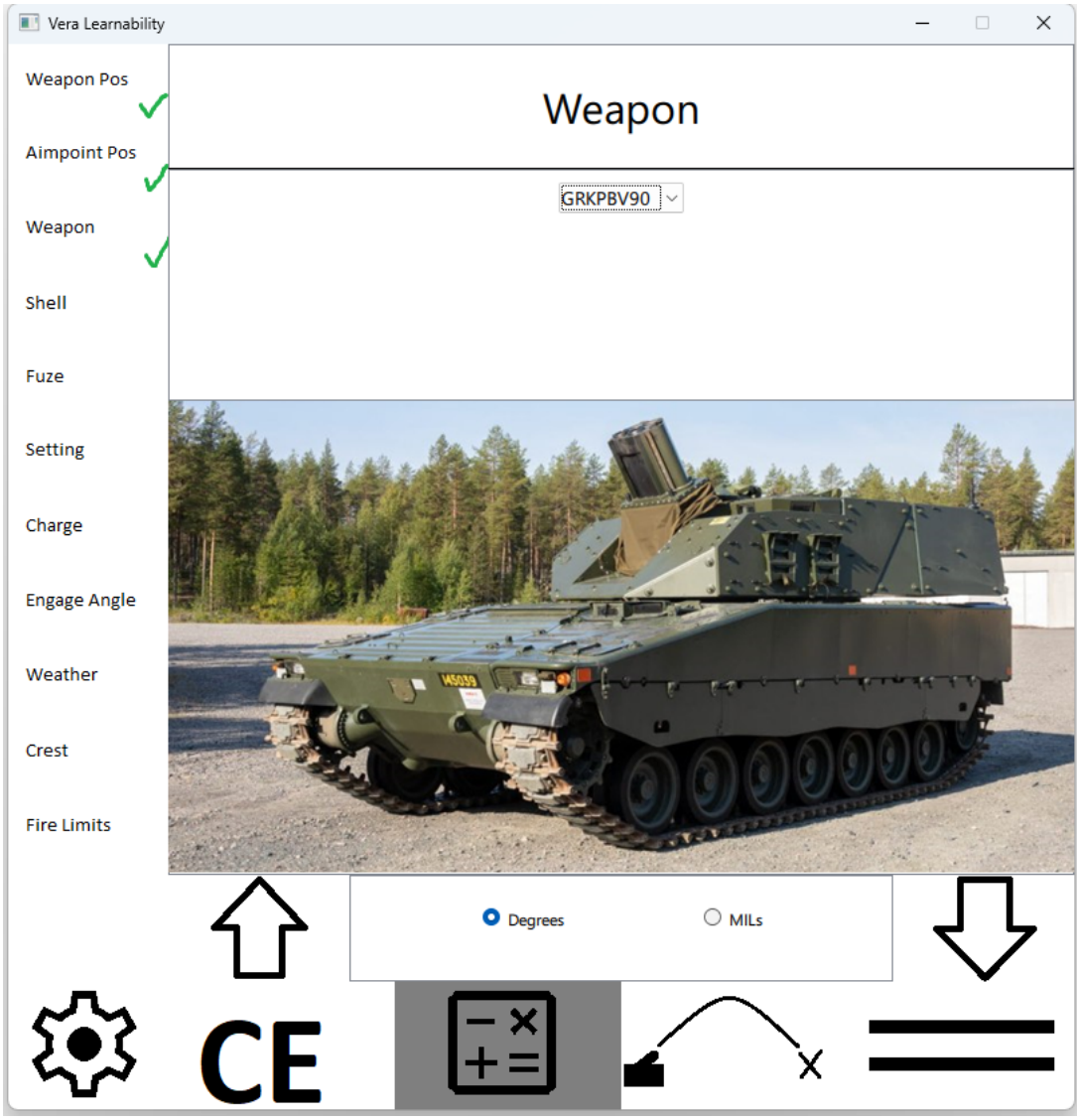


Figure .9.2: Weapon selection view

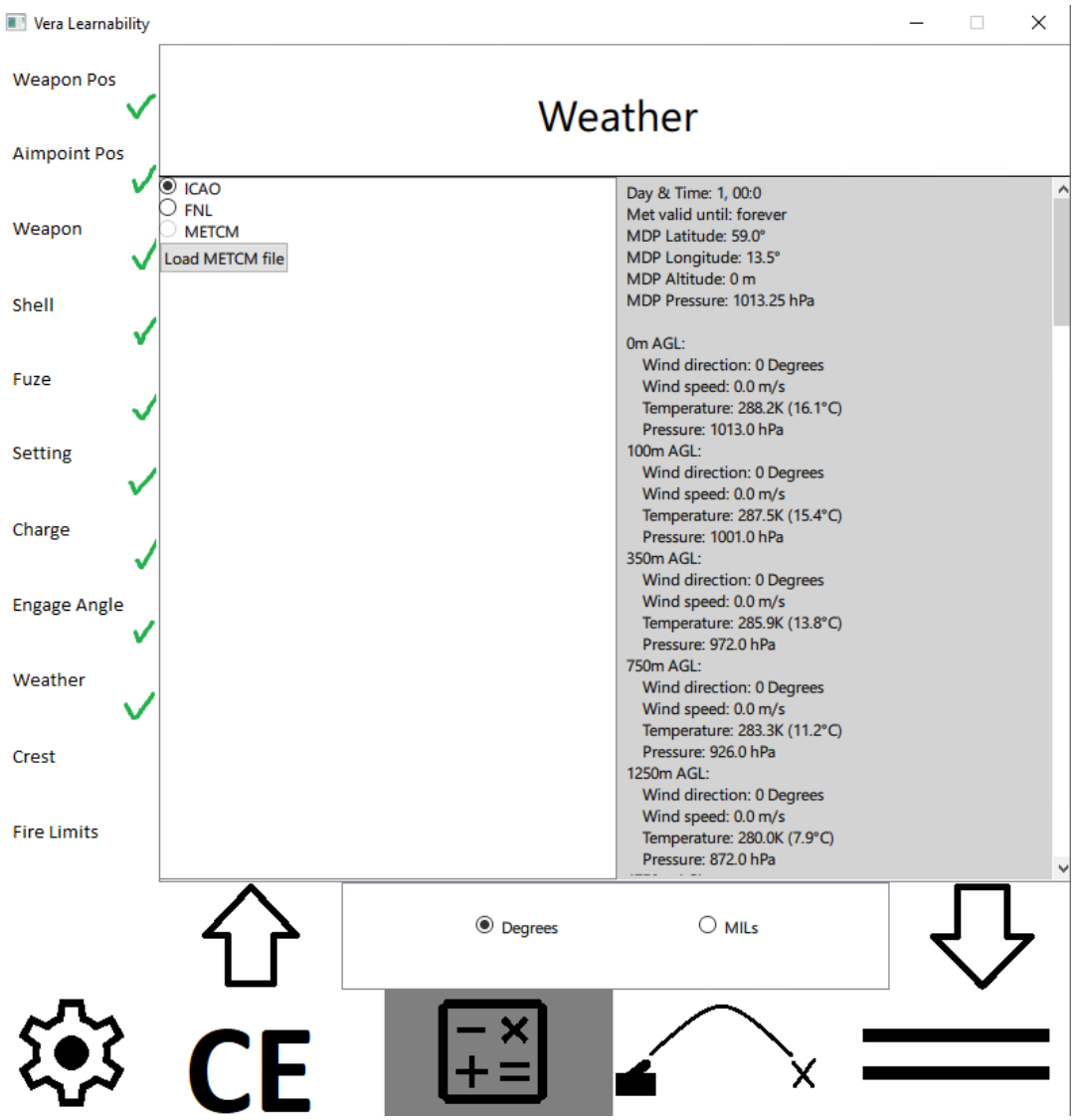


Figure .9.3: Weather selection view

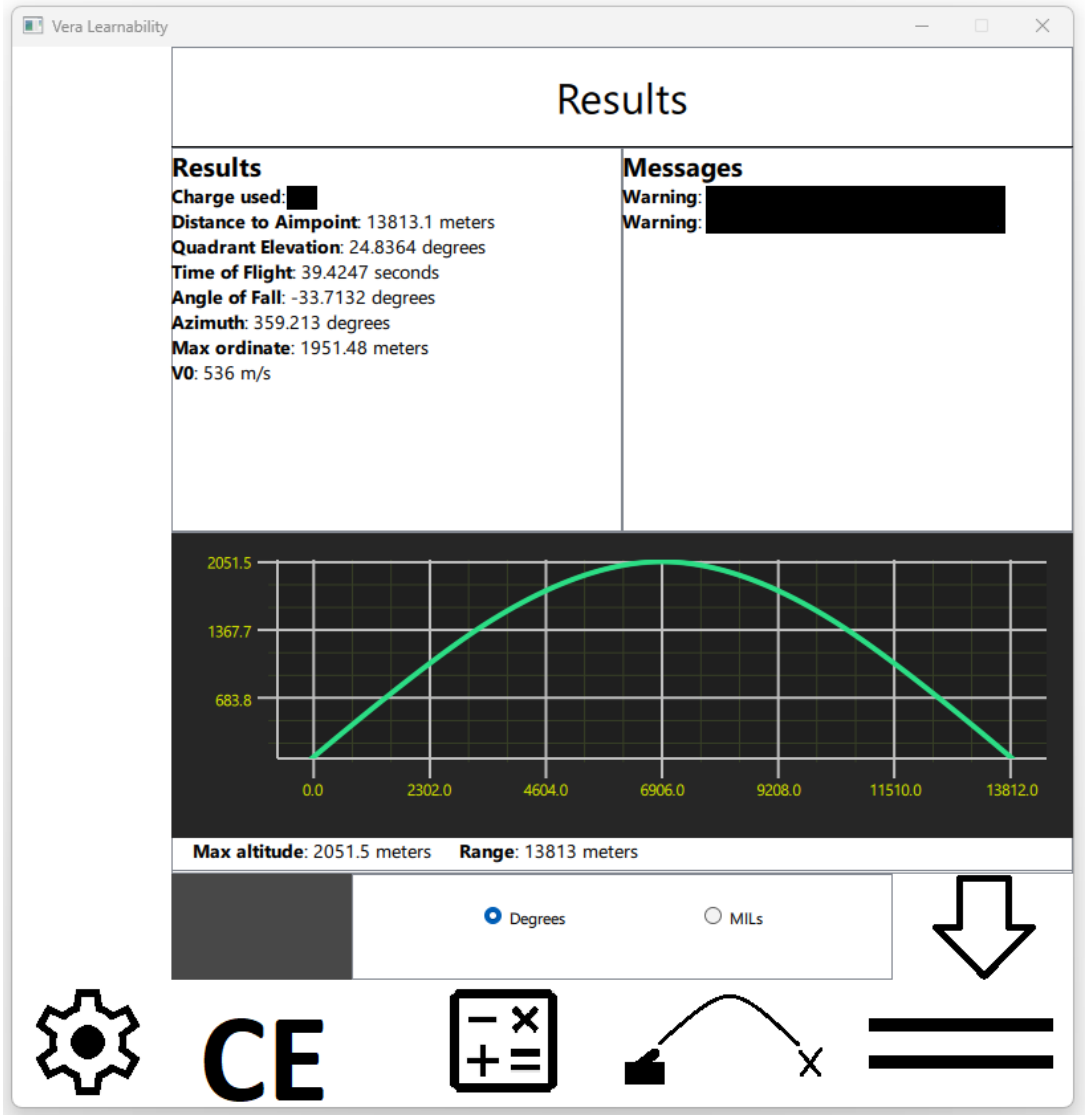


Figure .9.4: Results view